



On the Enumeration Complexity of Unions of Conjunctive Queries

NOFAR CARMELI, Technion - Israel Institute of Technology
MARKUS KRÖLL, TU Wien

We study the enumeration complexity of **Unions of Conjunctive Queries (UCQs)**. We aim to identify the UCQs that are tractable in the sense that the answer tuples can be enumerated with a linear preprocessing phase and a constant delay between every successive tuples. It has been established that, in the absence of self-joins and under conventional complexity assumptions, the CQs that admit such an evaluation are precisely the free-connex ones. A union of tractable CQs is always tractable. We generalize the notion of free-connexity from CQs to UCQs, thus showing that some unions containing intractable CQs are, in fact, tractable. Interestingly, some unions consisting of only intractable CQs are tractable too. We show how to use the techniques presented in this article also in settings where the database contains cardinality dependencies (including functional dependencies and key constraints) or when the UCQs contain disequalities. The question of finding a full characterization of the tractability of UCQs remains open. Nevertheless, we prove that, for several classes of queries, free-connexity fully captures the tractable UCQs.

CCS Concepts: • **Theory of computation** → *Database theory; Complexity classes; Database query languages (principles); Database query processing and optimization (theory);*

Additional Key Words and Phrases: Unions of conjunctive queries, enumeration, complexity, constant delay

ACM Reference format:

Nofar Carmeli and Markus Kröll. 2021. On the Enumeration Complexity of Unions of Conjunctive Queries. *ACM Trans. Database Syst.* 46, 2, Article 5 (May 2021), 41 pages.
<https://doi.org/10.1145/3450263>

1 INTRODUCTION

Evaluating a query Q over a database instance D is a fundamental and well-studied problem in database management systems. Most of the complexity results dealing with query evaluation so far aim to solve a decision or counting variant of it. Starting with Durand and Grandjean in 2007 [14], there has been a renewed interest in examining the enumeration problem of all answers to a query, focusing on fine-grained complexity [11, 16, 24, 25]. When evaluating a non-Boolean query over a database, the number of results may be larger than the size of the database itself. Enumeration complexity offers specific measures for the hardness of such problems. In terms of data complexity,

The work of Nofar Carmeli was supported by the Irwin and Joan Jacobs Fellowship and by the Google PhD Fellowship. The work of Markus Kröll was supported by the Austrian Science Fund (FWF): P30930-N35, W1255-N23.

Authors' addresses: N. Carmeli, Technion - Israel Institute of Technology, Haifa, Israel; email: snofca@cs.technion.ac.il; M. Kröll, Nofar Carmeli, CS Taub Building, Technion, Haifa 3200003, Israel; Markus Kröll, TU Wien, Favoritenstrasse 9-11/3, A-1040 Vienna, Austria; email: kroell@dbai.tuwien.ac.at.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

0362-5915/2021/05-ART5 \$15.00

<https://doi.org/10.1145/3450263>

the best time guarantee we can hope for is to output all answers with a constant delay between consecutive answers. In the case of query evaluation, this enumeration phase comes after a linear preprocessing phase required to read the database and decide the existence of a first answer. The enumeration class achieving these time bounds is denoted by $\text{DelayC}_{\text{lin}}$. Hereafter, we refer to queries in $\text{DelayC}_{\text{lin}}$ as tractable, and queries outside of this class as intractable.

Results from Bagan et al. [4] and Brault-Baron [9] form a dichotomy that fully classifies which self-join-free **Conjunctive Queries (CQs)** are in the class $\text{DelayC}_{\text{lin}}$ based on the structure of the query: the class of *free-connex* queries is exactly the class that admits tractable enumeration. In the years following this dichotomy, much work has been conducted to achieve similar results for other classes of queries [26]. Unions of CQs (UCQs) are a natural extension of CQs, as they describe the union of the answers to several CQs. UCQs form an important class of queries, as they capture the positive fragment of relational algebra. Previous work which implies results on the enumeration complexity of UCQs imposes strong restrictions on the underlying database [25, 27]. We aim to understand the enumeration complexity of UCQs without such restrictions and based solely on their structure.

Using known methods [28], it can be shown that a union of tractable problems is again tractable. However, what happens if some CQs of a union are tractable while others are not? Intuitively, one might be tempted to expect a union of enumeration problems to be harder than a single problem within the union, making such a UCQ intractable as well. As we will show, this is not necessarily the case.

Example 1. Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y) &\leftarrow R_1(x, y), R_2(y, z), R_3(z, x) \text{ and} \\ Q_2(x, y) &\leftarrow R_1(x, y), R_2(y, z). \end{aligned}$$

Even though Q_1 is hard while Q_2 is easy, a closer look shows that Q_2 contains Q_1 . This means that Q_1 is redundant, and the entire union is equivalent to the easy Q_2 .

To avoid cases like these, where the UCQ can be translated to a simpler one, it makes sense to consider non-redundant unions. It was claimed that, in all cases of a non-redundant union containing an intractable CQ, the UCQ is intractable too [8]. The following is a counter-example which refutes this claim.

Example 2. Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, w) &\leftarrow R_1(x, z), R_2(z, y), R_3(y, w) \text{ and} \\ Q_2(x, y, w) &\leftarrow R_1(x, y), R_2(y, w). \end{aligned}$$

According to the dichotomy of Bagan et al. [4], the enumeration problem for Q_2 is in $\text{DelayC}_{\text{lin}}$, while Q_1 is intractable. Yet, it turns out that Q is in fact in $\text{DelayC}_{\text{lin}}$. The reason is that, since Q_1 and Q_2 are evaluated over the same database I , we can use $Q_2(I)$ to find $Q_1(I)$. We can compute $Q_2(I)$ efficiently, and try to extend every such solution to solutions of Q_1 with a constant delay: for every new combination (a, c) of an output $(a, b, c) \in Q_2(I)$, we find all d values with $(b, d) \in R_3^I$ and then output the solution $(a, c, d) \in Q_1(I)$. The new combinations (a, c) can be detected by storing a lookup table containing (a, c) for every answer $(a, b, c) \in Q_2(I)$ generated. This table can be accessed in constant time in the RAM model. Avoiding duplicates in case Q_1 and Q_2 share answers requires also storing the previously seen answers, and it is discussed in more details in Section 3. Intuitively, the source of intractability for Q_1 is the join of R_1 with R_2 as we need to avoid duplicates that originate in different z values. The union is tractable since Q_2 returns exactly this join. $\square$

As the example illustrates, to compute the answers to a UCQ in an efficient way, it is not enough to view it as a union of isolated instances of CQ enumeration. In fact, this task requires an understanding of the interaction between several queries. Example 2 shows that the presence of an easy query within the union may give us enough time to compute auxiliary data structures, which we can then add to the hard queries in order to enumerate their answers as well. In Example 2, we can assume we have a ternary relation holding the result of Q_2 . Then, adding the auxiliary atom $R_{Q_2}(x, z, y)$ to Q_1 results in a tractable structure. We generalize this observation and introduce the concept of *union-extended* queries. We then use union extensions as a central tool for evaluating the enumeration complexity of UCQs, as the structure of such queries has implications on the tractability of the UCQ.

Interestingly, this approach can be taken a step further: We show that the concept of extending the union by auxiliary atoms can even be used to efficiently enumerate the answers of UCQs that *only* contain hard queries. By lifting the concept of free-connex queries from CQs to UCQs via union-extensions, we show that free-connex UCQs are always tractable. This gives us a sufficient global condition for membership in $\text{DelayC}_{\text{lin}}$ beyond any classification of individual CQs.

Finding a full characterization of the tractability of UCQs with respect to $\text{DelayC}_{\text{lin}}$ remains an open problem. Nevertheless, we prove that for several classes of queries, free-connexity fully captures the tractable UCQs. A non-free-connex union of two CQs is intractable in the following cases: both CQs are intractable, or they both represent the same CQ up to a different projection. The hardness results presented here use problems with well-established assumptions on the lower bounds, such as Boolean matrix-multiplication [21] or finding a clique or a hyperclique in a graph [22].

Why is establishing lower bounds on UCQ evaluation, even when it contains only two CQs, a fundamentally more challenging problem than the one for CQs? In the case of CQs, hardness results are often shown by reducing a computationally hard problem to the task of answering a query. The reduction encodes the hard problem to the relations of a self-join-free CQ, such that the answers of the CQ correspond to an answer of this problem [4, 7, 9, 11]. However, using such an encoding for CQs within a union does not always work. Similarly to the case of CQs with self-joins, relational symbols that appear multiple times within a query can interfere with the reduction. Indeed, when encoding a hard problem to an intractable CQ within a union, a different CQ in the union evaluates over the same relations, and may also produce answers. A large number of such supplementary answers, with constant delay per answer, accumulates to a long delay until we obtain the answers that correspond to the computationally hard problem. If this delay is larger than the lower bound we assume for the hard problem, we cannot conclude that the UCQ is intractable.

The lower bounds presented in this article are obtained either by identifying classes of UCQs for which we can use similar reductions to the ones used for CQs, or by introducing alternative reductions. As some cases remain unclassified, we devote a section to inspecting such UCQs, and describing the challenges that will need to be resolved in order to achieve a full classification.

Most of the results in this work refer to the classification of the hardness of UCQs over general database instances with respect to the time it takes to answer them with no space restrictions. Nonetheless, we also discuss the implications of these results when the settings are slightly modified. The first modification we consider is when the database instances are not general, as they are restricted by the schema. *Cardinality dependencies* are commonly given as part of the database schema to capture the fact that, since we know what our database represents, we know of restrictions on the combinations of values that the relations may hold. Functional Dependencies and key constraints are a special case of cardinality dependencies. In the presence of cardinality dependencies, additional CQs are tractable [11]. We show that by combining the approach introduced in this article with the one for identifying tractable CQs in the presence of cardinality dependencies, we can also find additional tractable UCQs over such schemas.

We also inspect UCQs with disequalities. In the case of CQs with disequalities, the disequalities have no effect on the enumeration complexity [4]. That is, one can simply ignore the disequalities, and the remaining CQ is free-connex if and only if the query is tractable (under the same assumptions as in Theorem 3). This irrelevance of disequalities to the enumeration complexity holds also in the case that there are cardinality dependencies in the schema [11]. A natural question is: Does this happen also with UCQs? We show that the answer is no: a tractable UCQ may become intractable when adding disequalities. We then show how to identify easy UCQs with disequalities using the same techniques we introduce for UCQs without them.

The final modification we consider is that of restricted space. We introduce the time complexity measure of *linear partial time*, which is more relaxed compared to linear preprocessing and constant delay, and discuss the connection between the two measures. We argue that from a theoretical point of view, when given access to enough space, the two time complexity measures are equivalent, and that from a practical point of view, an algorithm with only the relaxed requirements is sometimes preferable. We discuss the space consumption used in our approach, and demonstrate that in some cases it can be reduced to constant additional space. The question of when exactly this can be done remains open, but we claim in favor of using the measure of linear partial time as an intermediate step for future research.

This article is organized as follows: In Section 2, we provide definitions and state results that we use. Section 3 formalizes how CQs within a union can make each other easier, defines free-connex UCQs, and proves that free-connex UCQs are in DelayC_{lin}. In Section 4, we prove conditional lower bounds and conclude a dichotomy for some classes of UCQs. Section 5 discusses the future steps required for a full classification, and demonstrates examples of queries of unknown complexity. Section 6 inspects the implications of this work with modified settings: in the presence of cardinality dependencies, with disequalities and with restricted space. Concluding remarks are given in Section 7.

A short version of this article appeared in the *Proceedings of the 38th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS19)* [12]. This article has the following additions. First, it closes some gaps by presenting full proofs to the results stated in the conference version. Second, it includes Section 6, which discusses the implications of this work in additional settings.

2 PRELIMINARIES

In this section, we provide preliminary definitions as well as state results that we will use throughout this article.

Unions of Conjunctive Queries. A *schema* $\mathcal{S}$ is a set of *relational symbols* $\{R_1, \dots, R_n\}$. We denote the *arity* of a relational symbol R_i as $\text{arity}(R_i)$. Let dom be a finite set of constants. A database I over schema $\mathcal{S}$ is called an *instance* of $\mathcal{S}$, and it consists of a finite relation $R_i^I \subseteq \text{dom}^{\text{arity}(R_i)}$ for every relational symbol $R_i \in \mathcal{R}$.

Let var be a set of variables disjoint from dom . A *Conjunctive Query* (CQ) over schema $\mathcal{S}$ is an expression of the form $Q(\vec{p}) \leftarrow R_1(\vec{v}_1), \dots, R_m(\vec{v}_m)$, where $R_1, \dots, R_m$ are relational symbols of $\mathcal{S}$, the tuples $\vec{p}, \vec{v}_1, \dots, \vec{v}_m$ hold variables, and every variable in $\vec{p}$ appears in at least one of $\vec{v}_1, \dots, \vec{v}_m$. We often denote this query as Q . Define the variables of Q as $\text{var}(Q) = \bigcup_{i=1}^m \vec{v}_i$, and define the *free variables* of Q as $\text{free}(Q) = \vec{p}$. The variables of Q that are not free are called *existential*. We call $Q(\vec{p})$ the head of Q , and the atomic formulas $R_i(\vec{v}_i)$ are called *atoms*. We further use $\text{atoms}(Q)$ to denote the set of atoms of Q . We denote by $\text{full}(Q)$ the full version of a CQ Q obtained by adding all variables to the head of Q . A CQ is said to be *self-join-free* if no relational symbol appears in more than one atom.

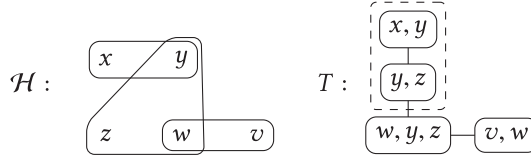


Fig. 1. T is an $\text{ext-}\{x, y, z\}$ -connex tree for hypergraph $\mathcal{H}$.

The *evaluation* $Q(I)$ of a CQ Q over a database I is the set of all *mappings* $\mu|_{\text{free}(Q)}$ such that μ is a homomorphism from $R_1(\vec{v}_1), \dots, R_m(\vec{v}_m)$ into I , and $\mu|_{\text{free}(Q)}$ is the restriction (or projection) of μ to the variables $\text{free}(Q)$. We refer to such mappings as *answers* to Q over I . To ease notation, we sometimes specify an answer as a tuple $\vec{u}$, and mean that the answer is the mapping that maps $\text{free}(Q)$ to $\vec{u}$. A CQ Q_1 is *contained* in a CQ Q_2 , denoted $Q_1 \subseteq Q_2$, if for every instance I , $Q_1(I) \subseteq Q_2(I)$. A *homomorphism* from Q_2 to Q_1 is a mapping $h : \text{var}(Q_2) \rightarrow \text{var}(Q_1)$ such that: (1) for every atom $R(\vec{v})$ of Q_2 , $R(h(\vec{v}))$ is an atom in Q_1 ; (2) $h(\text{free}(Q_2)) = \text{free}(Q_1)$.

A *Union of Conjunctive Queries* (UCQ) Q is a finite set of CQs, denoted $Q = \bigcup_{i=1}^{\ell} Q_i$, where $\text{free}(Q_{i_1}) = \text{free}(Q_{i_2})$ for all $1 \leq i_1, i_2 \leq \ell$. Semantically, $Q(I) = \bigcup_{i=1}^{\ell} Q_i(I)$. We say that Q is *non-redundant* if there are no two different CQs Q_1 and Q_2 in Q such that there is a homomorphism from Q_2 to Q_1 . We assume that UCQs are non-redundant; otherwise, an equivalent non-redundant UCQ can be obtained by removing the redundant CQs. Given a UCQ Q and a database instance I , we denote by $\text{DECIDE}(Q)$ the problem of deciding whether $Q(I) \neq \emptyset$.

Hypergraphs. A *hypergraph* $\mathcal{H} = (V, E)$ is a set V of *vertices* and a set E of non-empty subsets of V called *hyperedges* (sometimes *edges*). A *join tree* of a hypergraph $\mathcal{H} = (V, E)$ is a tree T where the nodes are the hyperedges of $\mathcal{H}$, and the *running intersection* property holds, namely: for all $u \in V$, the set $\{e \in E \mid u \in e\}$ forms a connected subtree in T . A hypergraph $\mathcal{H}$ is *acyclic* if there exists a join tree for $\mathcal{H}$.

Two vertices in a hypergraph are *neighbors* if they appear in a common edge. A *cycle* in a hypergraph is a sequence of vertices that starts and ends in the same vertex and every two consecutive vertices are neighbors. A hypergraph is *ischordal* if every cycle with at least four distinct vertices has an edge connecting two nonconsecutive vertices of the cycle. Every acyclic graph is chordal [5].

A *clique* of a hypergraph is a set of vertices, which are pairwise neighbors in $\mathcal{H}$. If every edge in $\mathcal{H}$ has exactly k vertices, we call $\mathcal{H}$ *k-uniform*. An *l-hyperclique* in a k -uniform hypergraph $\mathcal{H}$ is a set V' of $l > k$ vertices, such that every subset of V' of size k forms a hyperedge.

A hypergraph $\mathcal{H}'$ is an *inclusive extension* of $\mathcal{H}$ if every edge of $\mathcal{H}$ appears in $\mathcal{H}'$, and every edge of $\mathcal{H}'$ is a subset of some edge in $\mathcal{H}$. A tree T is an *ext-S-connex tree* for a hypergraph $\mathcal{H}$ if: (1) T is a join-tree of an inclusive extension of $\mathcal{H}$, and (2) there is a subtree T' of T that contains exactly the variables S [4] (see Figure 1).

Classes of CQs. We associate a hypergraph $\mathcal{H}(Q) = (V, E)$ to a CQ Q where the vertices are the variables of Q , and every hyperedge is a set of variables occurring in a single atom of Q . That is, $E = \{\{v_1, \dots, v_n\} \mid R_i(v_1, \dots, v_n) \in \text{atoms}(Q)\}$. With a slight abuse of notation, we identify atoms of Q with edges of $\mathcal{H}(Q)$. A CQ Q is said to be *acyclic* if $\mathcal{H}(Q)$ is acyclic.

A CQ Q is *S-connex* if $\mathcal{H}(Q)$ has an ext- S -connex tree, and it is *free-connex* if it has an ext- $\text{free}(Q)$ -connex tree [4]. Equivalently, Q is free-connex if both Q and $(V, E \cup \{\text{free}(Q)\})$ are acyclic [9]. A *free-path* in a CQ Q is a sequence of variables $(x, z_1, \dots, z_k, y)$ with $k \geq 1$, such that: (1) $\{x, y\} \subseteq \text{free}(Q)$, (2) $\{z_1, \dots, z_k\} \subseteq V \setminus \text{free}(Q)$, and (3) It is a *chordless path* in $\mathcal{H}(Q)$: that is, every two succeeding variables are neighbors in $\mathcal{H}(Q)$, but no two non-succeeding variables are neighbors. An acyclic CQ has a free-path iff it is not free-connex [4].

Computational Model. In this article, we adopt the *Random Access Machine* (RAM) model with uniform-cost measure and word length $\Theta(\log(n))$ on input of size n . Operations such as addition of the values of two registers or concatenation can be performed in constant time. In contrast to the Turing model of computation, the RAM model with uniform-cost measure can retrieve the content of any register via its unique address in constant time.

The input to our problems is a database instance I . We denote by $||o||$ the size of an object o (i.e., the number of integers required to store it), whereas $|o|$ is its cardinality. Let I be a database over a schema $S = (\mathcal{R}, \Delta)$. We assume the input database is given by the reasonable encoding suggested by Flum et al. [17]. Thus, the input is of size $||I|| = 1 + |\text{dom}| + |\mathcal{R}| + \sum_{R \in \mathcal{R}} \text{arity}(R) |R^I|$ over integers bounded by $\max\{|\text{dom}|, \max_{R \in \mathcal{R}} |R^I|\}$.

As we assume that the word length is $\Theta(\log(||I||))$, the values stored in registers are at most $||I||^c$ for some fixed integer c . As a consequence, and since we assume that the values may correspond to addresses, the amount of available memory is polynomial in $||I||$. Note that, even in more restrictive variants of the RAM model, the size of the registers and the available memory depend on the size of the input [18]. In particular, since we assume that the input fits in memory and each address fits in a register, the registers must contain at least $\log(n)$ bits for an input that requires n registers.

The RAM model enables the construction of large lookup tables that can be queried within constant time. If the available memory is as large as the number of different possible keys, the lookup table can be as simple as initializing the memory to False, and setting the addresses matching the keys to be True. Note that the time for initialization is not an issue due to constant time initialization techniques [23]. Solutions such as binary search trees and hash tables can reduce the memory consumption significantly. For example, a lookup table based on a binary search tree only requires linear memory in its content at the cost of a logarithmic factor to the time. In this work, we wish to identify what can and cannot be done within certain time bounds without restrictions on the memory consumption. Thus, we assume in our analysis that the available memory is very large and that we can access lookup tables of polynomial size in constant time.

Enumeration Complexity. Given a finite alphabet Σ and binary relation $R \subseteq \Sigma^* \times \Sigma^*$, the *enumeration problem* $\text{ENUM}\langle R \rangle$ is: given an instance $x \in \Sigma^*$, output all $y \in \Sigma^*$ such that $(x, y) \in R$. Such y values are often called *solutions* or *answers* to $\text{ENUM}\langle R \rangle$. An *enumeration algorithm* $\mathcal{A}$ for $\text{ENUM}\langle R \rangle$ is a RAM that solves $\text{ENUM}\langle R \rangle$ without repetitions. We say that $\mathcal{A}$ enumerates $\text{ENUM}\langle R \rangle$ with *delay* $d(|x|)$ if the time before the first output, the time between any two consecutive outputs, and the time between the last output and termination are each bounded by $d(|x|)$. Sometimes we wish to relax the requirements of the delay before the first answer, and specify a *preprocessing time* $p(|x|)$. In this case, the time before the first output is only required to be bounded by $p(|x|)$. The enumeration class $\text{DelayC}_{\text{lin}}$ is defined as the class of all enumeration problems $\text{ENUM}\langle R \rangle$ which have an enumeration algorithm $\mathcal{A}$ with preprocessing $p(|x|) \in O(|x|)$ and delay $d(|x|) \in O(1)$. Note that we do not impose a restriction on the memory used. In particular, such an algorithm may use additional constant memory for writing between two consecutive answers.

Let $\text{ENUM}\langle R_1 \rangle$ and $\text{ENUM}\langle R_2 \rangle$ be enumeration problems. There is an *exact reduction* from $\text{ENUM}\langle R_1 \rangle$ to $\text{ENUM}\langle R_2 \rangle$, denoted as $\text{ENUM}\langle R_1 \rangle \leq_e \text{ENUM}\langle R_2 \rangle$, if there exist mappings σ and τ such that: (1) for every $x \in \Sigma^*$, $\sigma(x)$ is computable in $O(|x|)$ time; (2) for every y such that $(\sigma(x), y) \in R_2$, $\tau(y)$ is computable in $O(1)$ time; and (3) in multiset notation, $\{\tau(y) \mid (\sigma(x), y) \in R_2\} = \{y' \mid (x, y') \in R_1\}$. Intuitively, σ maps instances of $\text{ENUM}\langle R_1 \rangle$ to instances of $\text{ENUM}\langle R_2 \rangle$, and τ maps solutions of $\text{ENUM}\langle R_2 \rangle$ to solutions of $\text{ENUM}\langle R_1 \rangle$. If $\text{ENUM}\langle R_1 \rangle \leq_e \text{ENUM}\langle R_2 \rangle$ and $\text{ENUM}\langle R_2 \rangle \in \text{DelayC}_{\text{lin}}$, then $\text{ENUM}\langle R_1 \rangle \in \text{DelayC}_{\text{lin}}$ as well [4].

Computational Hypotheses. We use the following well-known hypotheses for lower bounds on certain computational problems:

MAT-MUL: Two Boolean $n \times n$ matrices cannot be multiplied in time $O(n^2)$. This problem is equivalent to the evaluation of the query $\Pi(x, y) \leftarrow A(x, z), B(z, y)$ over the schema $\{A, B\}$ where $A, B \subseteq \{1, \dots, n\}^2$. It is conjectured that this problem cannot be solved in $O(n^2)$ time, and the best algorithms today require $O(n^\omega)$ time, where $2.37 < \omega < 2.38$ is the matrix multiplication exponent [2].

Remark: MAT-MUL is the assumption originally used by Bagan et al. [4] to show the hardness for CQs. It was later suggested to use a sparse variant of this hypothesis to show the same results [6]. Here, we use the original variant, as the sparse variant is not enough to handle cases like Lemma 27.

HYPERCLIQUE: For all $k \geq 3$, finding a k -hyperclique in a $(k-1)$ -uniform hypergraph with n vertices is not possible in time $O(n^{k-1})$. When $k = 3$, this is the assumption that we cannot detect a triangle in a graph in $O(n^2)$ time [3]. When $k > 3$, this is a special case of the (ℓ, k) -Hyperclique Hypothesis [22], which states that, in a k -uniform hypergraph of n vertices, $n^{k-o(1)}$ time is required to find a set of ℓ vertices such that each of its subsets of size k forms a hyperedge. The HYPERCLIQUE hypothesis is sometimes called Tetra $\langle k \rangle$ [9].

4-CLIQUE: It is not possible to determine the existence of a 4-clique in a graph with n nodes in time $O(n^3)$. This is a special case of the k -Clique Hypothesis [22], which states that detecting a clique in a graph with n nodes requires $n^{\frac{\omega k}{3}-o(1)}$ time, where ω is again the matrix multiplication exponent.

Enumerating Answers to UCQs. Given a UCQ Q over some schema $\mathcal{S}$, we denote by $\text{ENUM}\langle Q \rangle$ the enumeration problem $\text{ENUM}\langle R \rangle$, where R is the binary relation between instances I over $\mathcal{S}$ and sets of mappings $Q(I)$. We consider the size of the query as well as the size of the schema to be fixed. In the case of CQs, Bagan et al. [4] showed that a self-join-free acyclic CQ is in $\text{DelayC}_{\text{lin}}$ iff it is free-connex. In addition, Brault-Baron [9] showed that self-join-free cyclic queries are not in $\text{DelayC}_{\text{lin}}$. In fact, the existence of a single answer to a cyclic CQ cannot be determined in linear time. We call a CQ *difficult* if it is self-join-free and not free-connex. Difficult CQs are intractable according to the following dichotomy.

THEOREM 3 ([4, 9]). *Let Q be a self-join-free CQ.*

- (1) *If Q is free-connex, then $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$.*
- (2) *If Q is acyclic and not free-connex, then $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$, assuming MAT-MUL.*
- (3) *If Q is cyclic, then $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$, as $\text{DECIDE}\langle Q \rangle$ cannot be solved in linear time, assuming HYPERCLIQUE.*

The positive case of this dichotomy can be shown using the *Constant Delay Yannakakis* (CDY) algorithm [19]. It uses an ext-free(Q)-connex tree T for Q . First, it performs the classical Yannakakis preprocessing [29] over T to obtain a relation for each node in T , where all tuples can be used for some answer in $Q(I)$. Then, it considers only the subtree of T containing free(Q), and joins the relations corresponding to this subtree with constant delay.

3 UPPER BOUNDS VIA UNION EXTENSIONS

In this section, we identify tractable UCQs. Section 3.1 discusses unions that contain only tractable CQs. Section 3.2 inspects the requirements from UCQs in $\text{DelayC}_{\text{lin}}$ and proves the Cheater's Lemma: a tool that allows us to compile several enumeration algorithms into one. In Section 3.3, we introduce the concepts of *union extensions* and variable sets that a CQ can *supply* in order to help the evaluation of another CQ in the union. We generalize the notion of free-connexity to UCQs and show that such queries are in $\text{DelayC}_{\text{lin}}$ in Section 3.4.

3.1 Unions of Tractable CQs

Using known techniques [28, Proposition 2.38] a union of tractable CQs is also tractable.

THEOREM 4. *Let $Q = Q_1 \cup \dots \cup Q_n$ be a UCQ for some fixed $n \geq 1$. If all CQs in Q are free-connex, then $\text{ENUM}(Q) \in \text{DelayC}_{\text{lin}}$.*

PROOF. Algorithm 1 evaluates a union of two CQs. In case of a union $Q = \bigcup_{i=1}^{\ell} Q_i$ of more CQs, we can use this recursively by treating the second query as $Q_2 \cup \dots \cup Q_{\ell}$.

ALGORITHM 1: Answering a Union of Two Tractable CQs

```

1: while  $a \leftarrow Q_1(I).next()$  do
2:   if  $a \notin Q_2(I)$  then
3:     print  $a$ 
4:   else
5:     print  $Q_2(I).next()$ 
6:   while  $a \leftarrow Q_2(I).next()$  do
7:     print  $a$ 

```

By the end of the run, the algorithm prints $Q_1(I) \setminus Q_2(I)$ over all iterations of line 3, and it prints $Q_2(I)$ in lines 5 and 7. Line 5 is called $Q_1(I) \cap Q_2(I)$ times, so the command $Q_2(I).next()$ always succeeds there. For free-connex CQs after linear preprocessing time, the answers can be enumerated in constant delay, and testing whether a given mapping is an answer can be done in constant time [6]. Thus, this algorithm runs within the required time bounds. $\square$

The technique presented in the proof of Theorem 4 has the advantage that it does not require more than constant memory available for writing in the enumeration phase. Alternatively, this theorem is a consequence of the following lemma, which we prove next and gives us a general approach to compile several enumeration algorithms into one.

3.2 The Cheater's Lemma

In this section, we prove a lemma that is useful to show upper bounds for UCQs even in cases not covered by Theorem 4. To get a clearer notion of what it means for a problem to be in the class $\text{DelayC}_{\text{lin}}$, we first define linear partial time.

Definition 5 (Linear Partial Time). An algorithm runs in *linear partial time* if, for every input x , the time before the n th output is $O(|x| + n)$.

We claim next that if we relax the requirement of linear preprocessing and constant delay to allow a constant number of linear delay steps, we get linear partial time.

PROPOSITION 6. *Let $\mathcal{A}$ be an algorithm. If there exist constants a, b and c such that, on any input x , the time between successive outputs of $\mathcal{A}$ is bounded by $a(|x| + n)$ at most b times, where n is the number of answers printed up to that point, and bounded by c otherwise, then $\mathcal{A}$ runs in linear partial time.*

PROOF. Before the n th answer, there are at most b steps with delay larger than constant, and in these cases the delay is bounded by $a(|x| + n)$. Thus, the time in which the n th answer is produced is bounded by $ba(|x| + n) + cn \leq (ab + c)(|x| + n)$ for every n . $\square$

If space is not restricted (as in our case), this relaxation in the phrasing of the requirements does not change the requirements themselves, as we show that any algorithm that runs in linear partial

time can be modified to achieve linear preprocessing and constant delay. This can be done using the known technique [10, Proposition 12] of delaying the results to regularize the delay. In fact, we can further relax the phrasing by allowing a constant number of duplicates per answer.

LEMMA 7 (THE CHEATER'S LEMMA). *Let P be an enumeration problem. The following are equivalent:*

- (1) $P \in \text{DelayC}_{\text{lin}}$.
- (2) *There exist an algorithm $\mathcal{A}$ and constants c and d such that: $\mathcal{A}$ outputs the solutions to P , the time before the n th answer is bounded by $c(|x| + n)$, and every result is produced at most d times.*

PROOF. As any algorithm that runs in linear preprocessing and constant delay also runs in linear partial time, the direction $1 \Rightarrow 2$ is trivial. We now show the opposite direction.

We describe an algorithm $\mathcal{A}'$ that simulates $\mathcal{A}$, stores all generated results to prevent duplicates and holds back generated results to regularize the delay. $\mathcal{A}'$ maintains a lookup table containing the results that $\mathcal{A}$ generated and a queue containing those that were not yet printed. Both are initialized as empty. $\mathcal{A}'$ calls $\mathcal{A}$. When $\mathcal{A}$ returns a result, $\mathcal{A}'$ checks the lookup table to determine whether it was found before. If it was not, the result is added to both the lookup table and the queue. Otherwise, it is ignored. Since $\mathcal{A}$ runs in linear partial time, there exists a constant c such that the n th answer to $\mathcal{A}$ is obtained after $c(|x| + n)$ operations. $\mathcal{A}'$ first performs $c|x|$ computation steps, and then after every cd computation steps, it outputs a result from the queue. The queue is never empty when used: $\mathcal{A}'$ returns its i th result after $c|x| + (cd)i = c(|x| + di)$ computation steps; At this time, $\mathcal{A}$ produced at least di results, which contain at least i unique results. When it is done simulating $\mathcal{A}'$, $\mathcal{A}$ outputs all remaining results in the queue. By definition, $\mathcal{A}'$ operates with linear preprocessing time and constant delay. It outputs all results of $\mathcal{A}$ with no duplicates since, due to the lookup table, every result enters the queue exactly once. $\square$

The Cheater's Lemma formalizes how much we are allowed to "cheat" in order to show that a problem can be solved with linear preprocessing and constant delay by only showing an algorithm with relaxed requirements. To show that a problem is in $\text{DelayC}_{\text{lin}}$, it suffices to find an algorithm for this problem where the delay is usually constant, but it may be linear a constant number of times, and the number of times every result is produced is bounded by a constant. The allowed linear delay is not only with respect to the input, but also with respect to the already produced answers.

3.3 Union Extensions

As Example 2 shows, Theorem 4 does not cover all tractable UCQs. We now define union extensions and address the other cases. We first define *body-homomorphisms* between CQs to have the standard meaning of homomorphism, but without the restriction on the heads of the queries.

Definition 8. Let Q_1, Q_2 be CQs.

- A *body-homomorphism* from Q_2 to Q_1 is a mapping $h : \text{var}(Q_2) \rightarrow \text{var}(Q_1)$ such that for every atom $R(\vec{v})$ of Q_2 , $R(h(\vec{v})) \in Q_1$.
- If there exists a body-homomorphism h from Q_2 to Q_1 and vice-versa, we say that Q_1 and Q_2 are *body-homomorphically equivalent*.
- A *body-isomorphism* from Q_2 to Q_1 is a bijective mapping h such that h is a body-homomorphism from Q_2 to Q_1 and h^{-1} is a body-homomorphism from Q_1 to Q_2 .
- If there is a body-isomorphism between Q_2 and Q_1 , we say that Q_1 and Q_2 are *body-isomorphic*.

The standard definition of a homomorphism from Q_2 to Q_1 is then a body-homomorphism h from Q_2 to Q_1 such that $h(\text{free}(Q_2)) = \text{free}(Q_1)$. To demonstrate the definition of a body-homomorphism, consider the UCQ $Q = Q_1 \cup Q_2$ from Example 2 with the CQs $Q_1(x, y, w) \leftarrow R_1(x, z), R_2(z, y), R_3(y, w)$ and $Q_2(x, y, w) \leftarrow R_1(x, y), R_2(y, w)$. There is a body-homomorphism $h : Q_2 \rightarrow Q_1$ with $h(x) = x$, $h(y) = z$, and $h(w) = y$. However, this is not a body-isomorphism. Body-isomorphic CQs have the same body up to variable renaming (and the renaming is given by the body-isomorphism). Note that body-isomorphic CQs are necessarily body-homomorphically equivalent. The opposite direction holds for self-join-free CQs; that is, if two CQs are self-join-free and body-homomorphically equivalent, then they are necessarily body-isomorphic. We now formalize the way that one CQ can help with evaluating another CQ in the union by supplying variables.

Definition 9. We say that a CQ Q *supplies* a set V of variables if there exists S such that $V \subseteq S \subseteq \text{free}(Q)$ and Q is S -connex.¹

When considering again Example 2, the query Q_2 supplies $\{x, y, w\} \subseteq \text{free}(Q_2)$ as Q_2 is $\{x, y, w\}$ -connex. The following lemma shows why body-homomorphisms and supplying variables play an important role in UCQ enumeration. In case of a body-homomorphism from a supplying CQ Q_2 to another CQ Q_1 , we can produce an auxiliary relation that contains all possible value combinations of the matching variables in Q_1 . This can be done efficiently while producing some answers to Q_2 .²

LEMMA 10. *Let Q_2 be a CQ that supplies the variables $\vec{v}_2$. Given an instance I , one can compute with linear-time preprocessing and constant delay a set of mappings M from $\text{free}(Q_2)$ to the domain such that:*

- $M \subseteq Q_2(I)$
- M can be translated in time $O(|M|)$ to a relation R^M such that: for every CQ Q_1 with a body-homomorphism h from Q_2 to Q_1 and for every answer $\mu_1 \in \text{full}(Q_1)(I)$, there is the tuple $\mu_1(h(\vec{v}_2)) \in R^M$.

PROOF. According to Definition 9, there exists $\vec{v}_2 \subseteq S \subseteq \text{free}(Q_2)$ such that Q_2 is S -connex. Take an ext- S -connex tree T for Q_2 , and perform the CDY algorithm [19] on Q_2 while treating S as the free variables. This results in a set N of mappings from the variables of S to the domain such that $N = Q_2(I)|_S$. Note that, as we mentioned following Theorem 3, the CDY algorithm has a preprocessing stage that removes dangling tuples and guarantees that, at its end, there is a relation for each vertex of the tree such that each tuple of such a relation can be used for some answer.

For every mapping $\mu \in N$, we extend it once to obtain a mapping from all variables of Q_2 as follows: Go over all vertices of T starting from the connected subtree containing S and treating a neighbor of an already treated vertex at every step. Consider a step where in its beginning μ is a homomorphism from a set S_1 , and we are treating an atom $R(\vec{v}, \vec{u})$ where $\vec{v} \subseteq S_1$ and $\vec{u} \cap S_1 = \emptyset$. We take some tuple in R of the form $(\mu(\vec{v}), \vec{t})$ and extend μ to also map $\mu(\vec{u}) = \vec{t}$. Such a tuple exists since the dangling tuples were removed. This extension takes constant time, and, in its end, we have that $\mu|_{\text{free}(Q_2)} \in Q_2(I)$. Using these extensions, we set $M = \{\mu^+|_{\text{free}(Q_2)} \mid \mu^+ \text{ as an extension of } \mu\}$.

¹The conference version of this article contains a definition of *providing* variables instead of that of *supplying* variables. The difference is that the supplied variables are specified in terms of the giving CQ, while the providing variables are specified in terms of the receiving CQ. We need the terminology to refer to the giving CQ to be able to specify a variable set that one CQ supplies and several CQs use, and so we introduced this minor change of naming with respect to the previous version in order to avoid confusion.

²The version of Lemma 10 as it was phrased in the conference version of this article was incorrect. In fact, it was too strong, and the slightly weaker version given here describes more accurately what we need.

$\mu \in N$). We have that $M \subseteq Q_2(I)$. We also have that $M|_S = N = Q_2(I)|_S$, and since $\vec{v}_2 \subseteq S$, this means that $M|_{\vec{v}_2} = Q_2(I)|_{\vec{v}_2}$. The mappings M are computed with linear preprocessing and constant delay as this is the complexity of the CDY algorithm and the manipulations we describe only require constant time per answer.

We define $R^M = \{\mu^+(\vec{v}_2) \mid \mu^+ \in M\}$. Since $M|_{\vec{v}_2} = Q_2(I)|_{\vec{v}_2}$, this is the same as $\{\mu_2(\vec{v}_2) \mid \mu_2 \in Q_2(I)\}$. Let Q_1 be a CQ such that there is a body-homomorphism h from Q_2 to Q_1 , and let $\mu_1 \in \text{full}(Q_1)(I)$. Since h is a body-homomorphism, for every atom $R(\vec{v})$ in Q_2 , $R(h(\vec{v}))$ is an atom in Q_1 . Since μ_1 forms an answer to the full Q_1 , for every such atom $\mu_1(h(\vec{v})) \in R^I$. This means that $\mu_1 \circ h|_{\text{free}(Q_2)}$ is an answer to Q_2 , so there exists $\mu_2 \in Q_2(I)$ such that $\mu_1 \circ h|_{\text{free}(Q_2)} = \mu_2$. By construction, $\mu_1(h(\vec{v}_2)) = \mu_2(\vec{v}_2) \in R^M$. $\square$

Note that if the mapping h is not a body-homomorphism, Lemma 10 does not hold in general.

Example 11. Consider $Q = Q_1 \cup Q_2$, which is a slight modification of Example 2, with

$$\begin{aligned} Q_1(x, y, w) &\leftarrow R_1(x, z), R_2(z, y), R_3(y, w) \text{ and} \\ Q_2(x, y, w) &\leftarrow R_1(x, y), R_2(y, w), R_4(y). \end{aligned}$$

Since R_4 is not a relational symbol in Q_1 , there is no body-homomorphism from Q_2 to Q_1 . If $R_4^I = \text{dom}$, then we can take the same approach as in Example 2, as the answers of Q_2 form $Q_1(I)|_{\{x, z, y\}}$. However, if R_4^I is smaller, this extra atom may filter the answers to Q_2 , and we do not obtain all of $Q_1(I)|_{\{x, z, y\}}$ in general. $\square$

During evaluation, a set of supplied variables can form an auxiliary relation, accessible by an auxiliary atom. We call the query with its auxiliary atoms a *union extension*.

Definition 12. Let $Q = Q_1 \cup \dots \cup Q_n$ be a UCQ. An *extension sequence* for Q is a sequence $Q^1, \dots, Q^N$ where $Q = Q^1$ and for all $1 < j \leq N$, Q^j is a UCQ of the form $Q^j_1 \cup \dots \cup Q^j_n$ such that the following holds. For some relational symbol R_j that does not appear in Q_{j-1} and a sequence $\vec{v}_j$ of variables supplied by a previous $Q^{\ell}_{p(j)}$ (i.e., $\ell \leq j$ and $1 \leq p(j) \leq n$) we have the following for all $i = 1, \dots, n$:

- $Q^j_i = Q^{j-1}_i$; or
- there is a body-homomorphism $h_{p(j), i}$ from $Q^1_{p(j)}$ to Q^j_i , and Q^j_i is obtained by adding the atom $R_j(h_{p(j), i}(\vec{v}_j))$ to Q^{j-1}_i .

If such an extension sequence exists, we call Q^N a *union extension* of Q^1 . Atoms that appear in Q^N but not in Q^1 are called *virtual atoms*.

Consider Example 2 again. We already established that Q_2 supplies $\{x, y, w\} \subseteq \text{free}(Q_2)$ and that there is a body-homomorphism $h : \text{var}(Q_2) \rightarrow \text{var}(Q_1)$ with $h((x, y, w)) = (x, z, y)$. Thus, Q has the union extension that contains Q_2 and $Q^+_1(x, y, w) \leftarrow R_1(x, z), R_2(z, y), R_3(y, w), R'(x, z, y)$, where Q^+_1 is obtained by adding the virtual atom $R'(x, z, y)$ to Q_1 .

3.4 Extension-Based Tractability

Union extensions can transform an intractable query to a free-connex one.

Definition 13. Let $Q = Q_1 \cup \dots \cup Q_n$ be a UCQ.

- Q_1 is said to be *union-free-connex* with respect to Q if it has a free-connex union extension.
- Q is *free-connex* if all CQs in Q are union-free-connex.

Note that the term free-connex for UCQs is a generalization of that for CQs: If a UCQ Q contains only one CQ, then Q is free-connex iff the CQ it contains is free-connex. We next show that tractability of free-connex queries also carries over to UCQs.

THEOREM 14. *Let Q be a UCQ. If Q is free-connex, then $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$.*

PROOF. We begin by sketching the proof. For answering Q , we describe an algorithm $\mathcal{A}$ which is comprised of two phases: a provision phase and a final results phase. In the provision phase, a free-connex union-extension is instantiated for each CQ in the union. In the final results phase, the answers of every CQ in the union are enumerated via their free-connex union-extensions. These answers can be enumerated efficiently using the CDY algorithm (see Theorem 3). We will use Lemma 10 to generate some answers to Q during the provision phase; this permits us to use more than linear time before the final results phase while still achieving an enumeration algorithm with only linear preprocessing time. We will use the Cheater's Lemma (Lemma 7) to remove duplicates and obtain the time bounds we want.

Initial Algorithm. Let $Q = Q_1 \cup \dots \cup Q_n$, and take an extension sequence $Q^1, \dots, Q^N$ where $Q^1 = Q$ and Q^N comprises of free-connex CQs. The provision phase consists of $N - 1$ provision steps. Let $I = I^1$ be the input database instance. During the j th provision step (with $1 < j \leq N$), we extend the database instance I_{j-1} into an instance I_j that matches Q^j . We use Lemma 10 to generate a set of answers $M_j \subseteq Q_{e(j)}^{j-1}(I_{j-1})$ while also computing a relation R^{M_j} . We set $(R_j)^{I_j} := R^{M_j}$. The other relations remain as they were; that is, $R^{I_j} = R^{I_{j-1}}$ for every relational symbol in the UCQ except for R_j . By the end of the provision phase, the algorithm computes an instance I_N that matches Q^N . Finally, we perform the final results phase where we compute $Q_i^N(I_N)$ for every Q_i in the union using the CDY algorithm.

Correctness. We prove that for all $1 \leq i \leq n$ and $1 \leq j \leq N$ we have that $Q_i(I) = Q_i^j(I_j)$ by induction on j . The base case trivially holds as $Q_i(I) = Q_i^1(I_1)$ by definition. Now consider the j th provision step. Intuitively, answers to an extended CQ are by definition all answers to its previous version that agree with some tuple in the new atom. Since the new atom contains a projection of the answers, the extension has exactly the same answers as its previous version. More formally, let $Q_{e(j)}$ be a CQ extended in step j . For every mapping μ , by definition of the extension we have that $\mu \in Q_{e(j)}^j(I_j)$ if and only if both $\mu \in Q_{e(j)}^{j-1}(I_{j-1})$ and $\mu(h(\vec{v}_j)) \in R_j^{I_j}$, and these two conditions hold if and only if $\mu \in Q_{e(j)}^{j-1}(I_{j-1})$ since for all $\mu \in Q_{e(j)}^{j-1}(I_{j-1})$ we have that $\mu(h(\vec{v}_j)) \in R_j^{I_j}$. This proves that $Q_{e(j)}^j(I_j) = Q_{e(j)}^{j-1}(I_{j-1})$. This shows that for every Q_i in the union, $Q_i^j(I_j) = Q_i^{j-1}(I_{j-1})$. By the induction hypothesis, $Q_i^{j-1}(I_{j-1}) = Q_i(I)$. This concludes the proof that $Q_i^j(I_j) = Q_i(I)$ for all $j \leq N$, and in particular, $Q^N(I_N) = Q(I)$. Since $Q^N(I_N) = Q(I)$, the final results phase computes all answers to Q . Since $Q_i(I) = Q_i^j(I_j)$ for all i and j , the mappings generated by Lemma 10 during the provision phase are also answers to Q .

The algorithm $\mathcal{A}$ we presented so far produces the results we want, but it is not a constant delay enumeration algorithm. It is left to show that $\mathcal{A}$ conforms to the conditions of the Cheater's lemma. This would mean that we can apply the lemma, and conclude that $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$.

Duplicates. Overall the algorithm produces results in N provision steps during the provision phase and in n CQ evaluation steps during the final results phase. The results produced at each individual step contain no duplicates. Therefore, every result appears at most $N + n$ times. Since N and n are constants, this is a constant number of duplicates per value.

Delay. Consider the j th provision step. It starts with a preprocessing of time $O(|I_{j-1}|)$ followed by a constant delay enumeration of a set M_j of answers. At its end, $O(|M_j|)$ time is required to

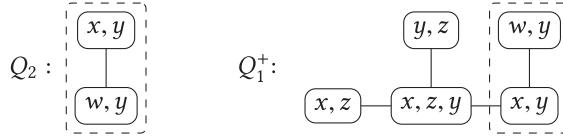


Fig. 2. A $\{x, y, w\}$ -connex tree for Q_2 and a $\{x, y, w\}$ -connex tree for Q_1^+ of Example 2.

compute the new relation. This means that the delay before the first answer of the j th provision step is $O(|I_{j-1}| + |M_{j-1}|)$, and the delay before answers in the provision phase that are not first in their step is constant. During the final results phase, we have n steps in which we apply the CDY algorithm on an extended CQ. Each such step starts with $O(|I_N|)$ preprocessing time followed by constant delay. Thus, during the final results phase, there are n times where the delay is $O(|I_N|)$, and in other times the delay is constant.

We have that $|I_j| = |I_{j-1}| + |R_j^{M_j}| \leq |I_{j-1}| + |M_j|$ for all $1 < j \leq N$, and $|I_1| = |I|$. By induction, this shows that $|I_j| \leq |I| + \sum_{i=2}^j |M_i|$. This means that whenever the delay is not constant, it is linear in the input size plus the number of (not necessarily unique) answers produces thus far. Since there are $n + N$ such times where the delay is not constant, according to Proposition 6, the algorithm $\mathcal{A}$ runs in linear partial time, and the requirements of the Cheater's Lemma on the delay are met. $\square$

We can now conclude the tractability of Example 2. In Section 3.3, we saw that Q_1 has the union extension $Q_1^+(x, y, w) \leftarrow R_1(x, z), R_2(z, y), R_3(y, w), R'(x, z, y)$. Since Q_1^+ is free-connex (see Figure 2), the CQ Q_1 is union-free-connex. Since every query in Q is union-free-connex, we have that $\text{ENUM}(Q) \in \text{DelayC}_{\text{lin}}$ by Theorem 14.

Remark 1. Example 2 is a counter example to a past made claim [8, Theorem 4.2b]. The claim is that if a UCQ contains an intractable CQ and does not contain redundant CQs (a CQ contained in another CQ in the union), then the union is intractable. In contrast, none of the CQs in Example 2 is redundant, Q_1 is intractable, and yet the UCQ is tractable.

The intuition behind the proof of the past claim is reducing the hard CQ Q_1 to Q . This can be done by assigning each variable of Q_1 with a different and disjoint domain (e.g., by concatenating the variable names to the values in the relations corresponding to the atoms), and leaving the relations that do not appear in the atoms of Q_1 empty. It is well known that $Q_1 \subseteq Q_2$ iff there exists a homomorphism from Q_2 to Q_1 [13]. The claim is that since there is no homomorphism from another CQ in the union to Q_1 , then there are no answers to the other CQs with this reduction. However, it is possible that there is a body-homomorphism from another CQ to Q_1 even if it is not a full homomorphism (the free variables do not map to each other). Therefore, in cases of a body-homomorphism, the reduction from Q_1 to Q does not work. In such cases, the union may be tractable, as we show in Theorem 14. In Lemma 18, we use the same proof described here, but restrict it to UCQs where there is no body-homomorphism from other CQs to Q_1 . $\square$

The tractability result in Theorem 14 is based on the structure of the union-extended queries. This means that the intractability of any query within a UCQ can be resolved as long as another query can supply the right variables. The following example shows that this can even be the case for a UCQ only consisting of non-free-connex CQs. It also illustrates why the definition of union extensions needs to be recursive.

Example 15. Let $Q = Q_1 \cup Q_2 \cup Q_3$ with

$$Q_1(x, y, v, u) \leftarrow R_1(x, z_1), R_2(z_1, z_2), R_3(z_2, z_3), R_4(z_3, y), R_5(y, v, u),$$

$$Q_2(x, y, v, u) \leftarrow R_1(x, y), R_2(y, v), R_3(v, z_1), R_4(z_1, u), R_5(u, t_1, t_2),$$

$$Q_3(x, y, v, u) \leftarrow R_1(x, z_1), R_2(z_1, y), R_3(y, v), R_4(v, u), R_5(u, t_1, t_2).$$

Each of three CQs is difficult on its own: Q_1 has the free-path (x, z_1, z_2, z_3, y) , while Q_2 has the free-path (v, z_1, u) , and Q_3 has the free-path (x, z_1, y) . The CQ Q_2 supplies $\{x, y, v\}$ as Q_2 is $\{x, y, v\}$ -connex and $\{x, y, v\}$ are free in Q_2 . Since there is a body-homomorphism $h_{2,3}$ from Q_2 to Q_3 with $h_{2,3}((x, y, v)) = (x, z_1, y)$, we can extend the body of Q_3 by the virtual atom $R'(x, z_1, y)$, which yields a free-connex extension Q_3^+ of Q_3 . Similarly, we have that Q_3 supplies $\{y, v, u\}$, and there is a body-homomorphism $h_{3,2}$ from Q_3 to Q_2 with $h_{3,2}((y, v, u)) = (v, z_1, u)$. Extending Q_2 by $R''(v, z_1, u)$ yields the free-connex extension Q_2^+ . Since Q_2^+ and Q_3^+ each supply $\{x, y, v, u\}$, we can add virtual atoms with the variables (x, z_1, z_2, y) and (x, z_2, z_3, y) to Q_1 . This results in a free-connex extension Q_1^+ . The UCQ $Q_1^+ \cup Q_2^+ \cup Q_3^+$ is a free-connex union extension of Q . By Theorem 14, $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$. $\square$

4 LOWER BOUNDS

In this section, we prove lower bounds for evaluating UCQs within the time bounds of $\text{DelayC}_{\text{lin}}$. Since for CQs we currently only have hardness result when they are self-join free, we focus on unions of self-join-free CQs. We begin with some general observations regarding cases where a UCQ is at least as hard as a single CQ it contains, and then continue to handle other cases. In Section 4.1 we discuss unions containing only difficult CQs, and in Section 4.2 we discuss unions containing two body-isomorphic CQs. In both cases such UCQs may be tractable, and in case of such a union of size two, we show that our results from Section 3 capture all tractable unions.

In order to provide some intuition for the choices we make throughout this section, we first explain where the approach used for proving the hardness of single CQs fails. Consider Example 2. The original proof that shows that Q_1 is hard describes a reduction from Boolean matrix multiplication [4, Lemma 26]. Let A and B be binary representations of Boolean $n \times n$ matrices, i.e., $(a, b) \in A$ corresponds to a 1 in the first matrix at index (a, b) . Define a database instance I as $R_1^I = A$, $R_2^I = B$, and $R_3^I = \{1, \dots, n\} \times \{\perp\}$. One can show that $Q_1(I)$ corresponds to the answers of AB . If $\text{ENUM}\langle Q_1 \rangle \in \text{DelayC}_{\text{lin}}$, we can solve matrix multiplication in time $O(n^2)$, in contradiction to MAT-MUL . Since Q_2 evaluates over the same relations, Q_2 also produces answers over this construction. Since the number of results for Q_2 might reach up to n^3 , evaluating Q in constant delay does not necessarily compute the answers to Q_1 in $O(n^2)$ time, and does not contradict the complexity assumption.

So, in general, whenever we show a lower bound to a UCQ by computing a hard problem through answering one CQ in the union, we need to ensure that the other CQs cannot have too many answers over this construction. The following lemma formalizes the idea that by assigning variables of a CQ with different domains, we can restrict the answers obtained by other CQs.

LEMMA 16. *Given a CQ Q_1 over a schema S , there exist mappings $\sigma_\#$ and $\tau_\#$ such that for every database instance I over S :*

- $Q_1(I) = \tau_\#(Q_1(\sigma_\#(I)))$ in multiset notation.
- $\sigma_\#(I)$ can be computed in linear time.
- For every CQ Q_i over S :
 - $\tau_\#(\mu)$ can be computed in constant time for every answer $\mu \in Q_i(\sigma_\#(I))$.
 - If there is no body-homomorphism from Q_i to Q_1 , $Q_i(\sigma_\#(I)) = \emptyset$.

—If there is no homomorphism from Q_i to Q_1 , given $\mu \in Q_1(\sigma_\#(I)) \cup Q_i(\sigma_\#(I))$, it is possible to determine in constant time whether $\mu \in Q_1(\sigma_\#(I))$.

PROOF. We define $\sigma_\#$ to assign each variable of Q_1 with a different and disjoint domain by concatenating the variable names to the values in their corresponding relations. For every atom $R(v_1, \dots, v_m)$ in Q_1 and tuple $(c_1, \dots, c_m) \in R^I$, we add the tuple $((c_1, v_1), \dots, (c_m, v_m))$ to $R^{\sigma_\#(I)}$. All relations that do not appear in Q_1 are left empty. We claim that the results of Q_1 over the original instance are exactly the same as over our construction if we omit the variable names. That is, we define $\tau_\# : \text{dom} \times \text{var}(Q_1) \rightarrow \text{dom}$ as $\tau_\#((c, v)) = c$, and show that $Q_1(I) = \tau_\#(Q_1(\sigma_\#(I)))$. Note the $\sigma_\#$ and $\tau_\#$ can be computed in linear and constant time, respectively.

We first prove that $Q_1(I) = \tau_\#(Q_1(\sigma_\#(I)))$. The first direction is trivial: if $v|_{\text{free}(Q_1)} \in Q_1(\sigma_\#(I))$, then for every atom $R(\vec{v})$ in Q_1 , $v(\vec{v}) \in R^{\sigma_\#(I)}$. By construction, $\tau_\#(v(\vec{v})) \in R^I$, and therefore $\tau_\# \circ v|_{\text{free}(Q_1)} \in Q_1(I)$. We now show the opposite direction. If $\mu|_{\text{free}(Q_1)} \in Q_1(I)$, then for every atom $R(v_1, \dots, v_m)$ in Q_1 , $(\mu(v_1), \dots, \mu(v_m)) \in R^I$. By construction, $((\mu(v_1), v_1), \dots, (\mu(v_m), v_m)) \in R^{\sigma_\#(I)}$. By defining $f_\mu : \text{var}(Q_1) \rightarrow \text{dom} \times \text{var}(Q_1)$ as $f_\mu(u) = (\mu(u), u)$, we have $f_\mu \in Q_1(\sigma_\#(I))$. Since $\tau_\# \circ f_\mu = \mu$, we have that $\mu|_{\text{free}(Q_1)} \in \tau_\#(Q_1(\sigma_\#(I)))$, and this concludes that $Q_1(I) \subseteq \tau_\#(Q_1(\sigma_\#(I)))$.

We now show that $Q_i(\sigma_\#(I)) = \emptyset$ if there is no body-homomorphism from Q_i to Q_1 . Assume by contradiction that there exists such $\mu|_{\text{free}(Q_i)} \in Q_i(\sigma_\#(I))$. This means that, for every atom $R(\vec{v})$ in Q_i , $\mu(\vec{v}) \in R^{\sigma_\#(I)}$. By construction, $\mu(\vec{v})$, like all tuples in $R^{\sigma_\#(I)}$, is of the form $((c_1, v_1), \dots, (c_m, v_m))$ such that $R(v_1, \dots, v_m)$ is an atom in Q_1 . Define $\eta : \text{dom} \times \text{var}(Q_1) \rightarrow \text{var}(Q_1)$ as $\eta(c, v) = v$. We have that for every atom $R(\vec{v})$ in Q_i , $R(\eta(\mu(\vec{v})))$ is an atom in Q_1 . This means that $\eta \circ \mu$ is a body-homomorphism from Q_i to Q_1 , which is a contradiction.

It is left to show that, if there is no homomorphism from Q_i to Q_1 , given $\mu \in Q_1(\sigma_\#(I)) \cup Q_i(\sigma_\#(I))$, it is possible to determine in constant time whether $\mu \in Q_1(\sigma_\#(I))$. As we showed, Q_i has answers over $\sigma_\#(I)$ only if there is a body-homomorphism h from it to Q_1 . Since this is not a full homomorphism, $h(\text{free}(Q_i)) \neq \text{free}(Q_1)$. Note that by definition of UCQs, the free variables are the same for all CQs in the union, and in particular $\text{free}(Q_i) = \text{free}(Q_1)$. If μ is an answer to Q_1 , then applying $\eta \circ \mu$ on $\text{free}(Q_1)$ will result in $\text{free}(Q_1)$. Otherwise (if μ is an answer to Q_i), then applying $\eta \circ \mu$ on $\text{free}(Q_1) = \text{free}(Q_i)$ will result in $h(\text{free}(Q_i))$. Since $h(\text{free}(Q_i)) \neq \text{free}(Q_1)$, this helps us distinguish the answers: apply $\eta \circ \mu$ on the $\text{free}(Q_1)$; if this results in $\text{free}(Q_1)$, then μ is an answer to Q_1 ; otherwise, μ is an answer to Q_i . $\square$

Example 17. Consider the union of $Q_1(x) \leftarrow R(x, y)$ with $Q_2(x) \leftarrow R(y, x)$ and $Q_3(x) \leftarrow R(x, x)$. If the input database I has $R^I = \{(a, a), (a, b)\}$, then all three queries have a as an answer, and Q_2 also has b . Applying the construction from Lemma 16 results in $R^{\sigma_\#(I)} = \{((a, x), (a, y)), ((a, x), (b, y))\}$. Then, we have that $Q_3(\sigma_\#(I)) = \emptyset$ as there is no body-homomorphism from Q_3 to Q_1 . There is a body-homomorphism from Q_2 to Q_1 but no such full homomorphism. We have that $Q_2(\sigma_\#(I)) = \{(a, y), (b, y)\}$ and $Q_1(\sigma_\#(I)) = \{(a, x)\}$, and so the answers can be distinguished by the concatenated variable name. Thus, if we are interested only in answers to Q_1 but we only have the ability to compute the entire UCQ, we can use this construction, ignore the answers containing y , and project out the variable names. This will result in the answer alone.

The following lemma identifies cases where we can encode any arbitrary instance of a CQ to an instance of the union containing it, such that no other CQ in the union returns results.

LEMMA 18. *Let Q be a UCQ, and let $Q_1 \in Q$ such that for all $Q_i \in Q \setminus \{Q_1\}$ there is no body-homomorphism from Q_i to Q_1 . Then $\text{ENUM}\langle Q_1 \rangle \leq_e \text{ENUM}\langle Q \rangle$.*

PROOF. This reduction can be performed using the mappings $\sigma_\#$ and $\tau_\#$ defined in Lemma 16. For every $Q_i \in Q \setminus \{Q_1\}$, there is no body-homomorphism from Q_i to Q_1 , and therefore $Q_i(\sigma_\#(I)) = \emptyset$. We now have that $\tau_\#(Q(\sigma_\#(I))) = \bigcup_{Q_i \in Q} \tau_\#(Q_i(\sigma_\#(I))) = \tau_\#(Q_1(\sigma_\#(I)))$. Since we also know that $\tau_\#(Q_1(\sigma_\#(I))) = Q_1(I)$, this concludes our reduction. $\square$

The lemma above implies that if there is an intractable CQ in a union where no other CQ maps to it via a body-homomorphism, then the entire union is intractable. This also captures cases such as a union of CQs where one of them is hard, and the others contain a relation that does not appear in the hard CQ.

Using the same reduction, a similar statement with relaxed requirements can be made in case it is sufficient to consider the decision problem.

LEMMA 19. *Let Q be a UCQ, and let $Q_1 \in Q$ such that for all $Q_i \in Q$, either there exists no body-homomorphism from Q_i to Q_1 , or Q_1 and Q_i are body-isomorphic. Then, $\text{DECIDE}\langle Q_1 \rangle \leq \text{DECIDE}\langle Q \rangle$ via a linear-time many-one reduction.*

PROOF. We use the encoding from Lemma 16. We know that $Q_i(\sigma_\#(I)) = \emptyset$ for every CQ Q_i with no body-homomorphism to Q_1 . We claim now that a CQ Q_j which is body-isomorphic to Q_1 has an answer iff Q_1 has an answer. Therefore, $Q(\sigma_\#(I)) \neq \emptyset$ iff $Q_1(\sigma_\#(I)) \neq \emptyset$. We know that Q_1 retains the same answers under this encoding, so in particular $Q_1(\sigma_\#(I)) = \emptyset$ if and only if $Q_1(I) = \emptyset$. Overall, this shows that $Q(\sigma_\#(I)) \neq \emptyset$ if and only if $Q_1(I) \neq \emptyset$.

We now show formally that, for body-isomorphic CQs Q_1 and Q_2 and database I , $Q_1(I) \neq \emptyset$ iff $Q_2(I) \neq \emptyset$. Let h be a body-homomorphism from Q_2 to Q_1 . That is, for every atom $R(\vec{v}) \in Q_2$, we have $R(h(\vec{v})) \in Q_1$. If $Q_1(I) \neq \emptyset$, then there exists $\mu|_{\text{free}(Q_1)} \in Q_1(I)$, and for every atom $R(h(\vec{v})) \in Q_1$, we have $\mu(h(\vec{v})) \in I$. This means that $\mu \circ h$ is a homomorphism from Q_2 to I , and $\mu \circ h|_{\text{free}(Q_2)} \in Q_2(I)$. So, $Q_2(I) \neq \emptyset$. Since there is also a body-homomorphism from Q_1 to Q_2 , we can show in the same way that if $Q_2(I) \neq \emptyset$ then $Q_1(I) \neq \emptyset$. $\square$

From Theorem 3, it follows that deciding whether a cyclic CQ has any answers cannot be done in linear time (assuming HYPERCLIQUE). Following Lemma 19, if a UCQ Q contains a cyclic Q_1 , and the conditions of Lemma 19 are satisfied with respect to this CQ, then the entire union cannot be decided in linear time, and thus $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$.

4.1 Unions of Intractable CQs

We now discuss unions containing only CQs classified as hard according to Theorem 3. Recall that these are called *difficult* CQs, and they are self-join-free CQs and not free-connex. The following lemma can be used to identify a CQ on which we can apply Lemma 18 or Lemma 19.

LEMMA 20. *Let Q be a UCQ. There exists a query $Q_1 \in Q$ such that for all $Q_i \in Q$ either there is no body-homomorphism from Q_i to Q_1 or Q_1 and Q_i are body-homomorphically equivalent.*

PROOF. Consider a longest sequence $(Q^1, \dots, Q^m)$ of CQs from Q such that for every $2 \leq j \leq m$ there is a body-homomorphism from Q^j to Q^{j-1} , but no body-homomorphism in the opposite direction. We claim that $Q_1 = Q^m$ satisfies the conditions of the lemma.

First, we show that such a sequence exists. We denote the body-homomorphism from Q^j to Q^{j-1} by μ^j . It is not possible that the same query appears twice in the sequence: if $Q^i = Q^j$ where $j > i$, then there is a mapping $\mu^{i+2} \circ \dots \circ \mu^j$ from $Q^j = Q^i$ to Q^{i+1} , in contradiction to the definition of the sequence. Therefore, $m \leq |Q|$, and a longest sequence exists. We now show that Q^m satisfies the requirements. First consider some $Q^j \in \{Q^1, \dots, Q^{m-1}\}$. There is a body-homomorphism from Q^m to Q^j which is the composition $\mu^{j+1} \circ \dots \circ \mu^m$. Therefore, either there is no body-homomorphism from Q^j to Q^m , or Q^m and Q^j are body-homomorphically equivalent.

In addition, Q^m is body-homomorphically equivalent to itself with the identity mapping serving as the body-homomorphisms. It is left to consider CQs that are not on the sequence. Let $Q_i \in Q \setminus \{Q^1, \dots, Q^m\}$. If there is no body-homomorphism from Q_i to Q^m , then we are done. Otherwise, if there is also no body-homomorphism from Q_1 to Q^m , then $(Q^1, \dots, Q^m, Q_i)$ is a longer sequence, contradicting the maximality of the sequence we started with. Therefore, in this case, Q^m and Q_i are body-homomorphically equivalent. Overall, we showed that for each query in Q either there is no body-homomorphism from this query to Q^m or these two CQs are body-homomorphically equivalent. $\square$

Using the results obtained so far, we deduce a characterization of all cases of a union of difficult CQs, except those that contain a pair of body-isomorphic acyclic CQs.

THEOREM 21. *Let Q be a UCQ of difficult CQs that does not contain two body-isomorphic acyclic CQs. Then, $Q \notin \text{DelayC}_{\text{lin}}$, assuming MAT-MUL and HYPERCLIQUE.*

PROOF. Let Q_1 be a CQ in Q given by Lemma 20. By definition, difficult CQs are self-join-free, and so if two CQs in Q are body-homomorphically equivalent, they are also body-isomorphic. We treat the two possible cases of the structure of Q_1 . In case Q_1 is acyclic, since we know that Q does not contain body-isomorphic acyclic CQs, then, for all $Q_i \in Q \setminus \{Q_1\}$, there is no body-homomorphism from Q_i to Q_1 . According to Lemma 18, $\text{ENUM}\langle Q_1 \rangle \leq_e \text{ENUM}\langle Q \rangle$. Since Q_1 is self-join-free acyclic non-free-connex, we have that $\text{ENUM}\langle Q_1 \rangle \notin \text{DelayC}_{\text{lin}}$ assuming MAT-MUL. Therefore $\text{ENUM}\langle Q \rangle$ is not in $\text{DelayC}_{\text{lin}}$ either. In case Q_1 is cyclic, we use Lemma 19 to conclude that $\text{DECIDE}\langle Q_1 \rangle \leq \text{DECIDE}\langle Q \rangle$. According to Theorem 3, since Q_1 is self-join-free cyclic, $\text{DECIDE}\langle Q_1 \rangle$ cannot be solved in linear time assuming HYPERCLIQUE. Therefore, $\text{DECIDE}\langle Q \rangle$ cannot be solved in linear time either, and $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$. $\square$

In the next example, we demonstrate how the reductions described in Lemma 19 and Theorem 3 combine in Theorem 21.

Example 22. Consider the UCQ $Q = Q_1 \cup Q_2 \cup Q_3$ with

$$\begin{aligned} Q_1(x, y) &\leftarrow R_1(x, y), R_2(y, u), R_3(x, u), \\ Q_2(x, y) &\leftarrow R_1(y, v), R_2(v, x), R_3(y, x), \\ Q_3(x, y) &\leftarrow R_1(x, z), R_2(y, z). \end{aligned}$$

The queries Q_1 and Q_2 are cyclic, and Q_3 is acyclic but not free-connex. This union is intractable according to Theorem 21. Note that Q_1 and Q_2 are body-isomorphic, but there is no body-homomorphism from Q_3 to Q_1 . The proof of Theorem 3 states the following: If $\text{ENUM}\langle Q_1 \rangle \in \text{DelayC}_{\text{lin}}$, then given an input graph G , we can use Q_1 to decide the existence of triangles in G in time $O(n^2)$, in contradiction to HYPERCLIQUE. The same holds true for $\text{ENUM}\langle Q \rangle$. For every edge (u, v) in G with $u < v$ we add $((u, x), (v, y))$ to R_1^I , $((u, y), (v, z))$ to R_2^I and $((u, x), (v, z))$ to R_3^I . The query detects triangles: for every triangle a, b, c in G with $a < b < c$, the query Q_1 returns $((a, x), (b, y))$. The union only returns answers corresponding to triangles:

- For every answer $((d, x), (e, y))$ to Q_1 , there exists some f such that d, e, f is a triangle in G with $d < e < f$.
- For every answer $((g, z), (h, x))$ to Q_2 , there exists some i such that g, h, i is a triangle in G with $h < i < g$.
- The query Q_3 returns no answers over I . $\square$

Theorem 21 does not cover the case of a UCQ containing acyclic non-free-connex queries with isomorphic bodies. Since this requires a more intricate analysis, we first restrict ourselves to such unions of size two.

4.2 Unions of Two Body-Isomorphic CQs

Consider a set of body-isomorphic CQs. As all of them have the same structure, either every CQ in this set is cyclic, or every CQ is acyclic. In the case of a union of two cyclic CQs, the UCQ is intractable according to Theorem 21. So in this section, we discuss the union of body-isomorphic acyclic CQs. Note that, unlike the previous section, we allow a CQ in the union to be free-connex. We first introduce a new notation for body-isomorphic UCQs that we use hereafter in order to simplify the discussion on the query structure.

Consider a UCQ of the form $Q_1 \cup Q_2$, where there exists a body-isomorphism h from Q_2 to Q_1 . That is, the CQs have the structure:

$$\begin{aligned} Q_1(\vec{v}_1) &\leftarrow R_1(h(\vec{w}_1)), \dots, R_n(h(\vec{w}_n)), \\ Q_2(\vec{v}_2) &\leftarrow R_1(\vec{w}_1), \dots, R_n(\vec{w}_n). \end{aligned}$$

By applying h^{-1} to the variables of Q_1 , we can rewrite Q_1 as $Q_1(h^{-1}(\vec{v}_1)) \leftarrow R_1(\vec{w}_1), \dots, R_n(\vec{w}_n)$. Every mapping μ that is an answer to Q_1 in its new form can easily be converted to an answer to the original Q_1 by composing it with h . So, for simplicity, we can assume that Q_1 is given in the modified form. Since now the two CQs have exactly the same body, we can treat the UCQ as a query with one body and two heads:

$$Q_1(h^{-1}(\vec{v}_1)), Q_2(\vec{v}_2) \leftarrow R_1(\vec{w}_1), \dots, R_n(\vec{w}_n).$$

We use this notation from now on for UCQs containing only body-isomorphic CQs. Note that, when treating a UCQ as one CQ with several heads, we can use the notation $\text{atoms}(Q)$, as the atoms are the same for all CQs in the union, and the notation $\text{free}(Q_i)$, as the free variables may differ between different queries Q_i in the union. With this notation at hand, we now inspect some examples of two body-isomorphic acyclic CQs.

Example 23. Consider $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, v) &\leftarrow R_1(x, z), R_2(z, y), R_3(y, v), R_4(v, w) \text{ and} \\ Q_2(x, y, v) &\leftarrow R_1(w, v), R_2(v, y), R_3(y, z), R_4(z, x). \end{aligned}$$

Since Q_1 and Q_2 are body-isomorphic, Q can be rewritten as

$$Q_1(w, y, z), Q_2(x, y, v) \leftarrow R_1(w, v), R_2(v, y), R_3(y, z), R_4(z, x).$$

In this case, we can use the same approach used for single CQs in Theorem 3, and show that this UCQ is not in $\text{DelayC}_{\text{lin}}$ assuming MAT-MUL . Let A and B be binary representations of Boolean $n \times n$ matrices as explained in the beginning of this section. Define a database instance I with $R_1^I = A$, $R_2^I = B$, $R_3^I = \{1, \dots, n\} \times \{\perp\}$ and $R_4^I = \{(\perp, \perp)\}$. $Q_1(I)$ corresponds to the answers of AB , and $|Q_2(I)| = O(n^2)$. Assume by contradiction that $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$. Then, we find $Q(I)$ in $O(n^2)$ time. We can distinguish the answers of Q_1 from those of Q_2 since the possible assignments to the variables in different queries are from disjoint domains: given an answer $\mu \in Q(I)$, we have that $\mu \in Q_1(I)$ if and only if $\mu(v) = \perp$. This means we can solve matrix multiplication in time $O(n^2)$, which is a contradiction to MAT-MUL . $\square$

A union of two difficult body-isomorphic acyclic CQs may also be tractable. In fact, by adding a single variable to the heads in Example 23, we obtain a tractable UCQ.

Example 24. Let Q be the UCQ

$$Q_1(w, y, x, z), Q_2(x, y, w, v) \leftarrow R_1(w, v), R_2(v, y), R_3(y, z), R_4(z, x).$$

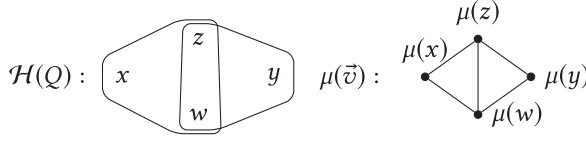


Fig. 3. If $\mu|_{\text{free}(Q_i)} \in Q(I)$, the induced subgraph of $\mu(w, x, y, z)$ forms a clique where one edge might be missing.

Both CQs are acyclic non-free-connex. As Q_2 supplies the variables $\{v, w, y\}$ and Q_1 supplies $\{x, y, z\}$, both CQs have free-connex union extensions:

$$\begin{aligned} Q_1^+(w, y, x, z) &\leftarrow R_1(w, v), R_2(v, y), R_3(y, z), R_4(z, x), P_1(v, w, y), \\ Q_2^+(x, y, w, v) &\leftarrow R_1(w, v), R_2(v, y), R_3(y, z), R_4(z, x), P_2(x, y, z). \end{aligned}$$

By Theorem 14, it follows that $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$. $\square$

Intuitively, the reason why the reduction of Example 23 fails in Example 24 is the fact that all the variables of the free-paths in one CQ, which are used to encode matrix multiplication, are free in the other CQ. Indeed, if we encode matrices A and B to the relations of the free path w, v, y in Q_1 , there can be n^3 answers to Q_2 . The answer set in this case is too large to contradict the assumed lower bound for matrix multiplication. As it turns out, there are cases where we cannot reduce matrix multiplication to a union in this manner, and yet we can show that it is intractable using an alternative problem.

Example 25. Let Q be the UCQ

$$Q_1(x, y, u), Q_2(x, y, z) \leftarrow R_1(x, u, z), R_2(y, u, z).$$

This union is intractable under the 4-CLIQUE assumption. For a given graph $G = (V, E)$ with $|V| = n$, we compute the set T of all triangles in G in time $O(n^3)$. Define a database instance I as $R_1^I = R_2^I = T$. For every output $\mu|_{\text{free}(Q_i)} \in Q(I)$ with $i \in \{1, 2\}$, we know that $(\mu(x), \mu(u), \mu(z))$ and $(\mu(y), \mu(u), \mu(z))$ are triangles. If $\mu(x) \neq \mu(y)$, this means that $\mu((x, y)) \in E$ if and only if $(\mu(x), \mu(y), \mu(u), \mu(z))$ forms a 4-clique (see Figure 3). Moreover, for every 4-clique (a, b, c, d) in the graph, we have $(a, b, c) \in Q_1(I)$, so this will detect all 4-cliques. Since there are $O(n^3)$ answers to Q , if $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$, we can check whether $\mu((x, y)) \in E$ for every answer in $Q(I)$, and determine whether a 4-clique appears in G in time $O(n^3)$. $\square$

Note that we can use the 4-CLIQUE assumption in Example 25, since, in addition to the free-path variables, there is another variable in both free-path relations. We now generalize the observations from the examples.

Definition 26. Let $Q = Q_1 \cup Q_2$ be a UCQ where Q_1 and Q_2 are body-isomorphic.

- Q_1 is said to be *free-path guarded* if for every free-path P in Q_1 , we have that $\text{var}(P) \subseteq \text{free}(Q_2)$.
- Consider a path $P = (u_1, \dots, u_k)$ in Q_1 . Two atoms $R_1(\vec{v}_1)$ and $R_2(\vec{v}_2)$ of Q_1 are called *subsequent P-atoms* if $\{u_{i-1}, u_i\} \subseteq \vec{v}_1$ and $\{u_i, u_{i+1}\} \subseteq \vec{v}_2$ for some $1 < i < k$.
- Q_1 is said to be *bypass guarded* if for every free-path P in Q_1 and variable u that appears in two subsequent P -atoms, we have that $u \in \text{free}(Q_2)$.

Note that every free-connex CQ is trivially free-path guarded and bypass guarded.

Let us demonstrate the definitions on the recent examples. The CQ Q_1 of Example 24 is both free-path guarded and bypass guarded. The only free-path it contains is $P = (w, v, y)$, and Q_1 is free-path guarded since $\{w, v, y\} \subseteq \text{free}(Q_2)$. The only subsequent P -atoms are $R_1(w, v)$ and $R_2(v, y)$,

and Q_1 is bypass guarded since $v \in \text{free}(Q_2)$. The query Q_1 of Example 23 is not free-path guarded as the variables of the free-path $P' = (w, v, y)$ of Q_1 are not contained in $\text{free}(Q_2)$. The query Q_1 of Example 25 is not bypass guarded. Consider the free-path $P'' = (x, z, y)$ of Q_1 . The atoms $R_1(x, u, z)$ and $R_2(y, u, z)$ are subsequent P'' atoms. Since u is contained in both atoms but not in $\text{free}(Q_2)$, we have that Q_1 is not bypass guarded.

In the following two lemmas, we prove that if some CQ in a union is either not free-path guarded or bypass guarded, then the UCQ is intractable. As in the characterization for CQs in Theorem 3, our lower bounds apply only when the CQs are self-join-free; with this restriction, we can assign different atoms with different relations. The next lemma shows that the reduction in Example 23, where we can use the fact that Q_1 is not free-path guarded to compute matrix multiplication, can be constructed in the general case as well.

LEMMA 27. *Let $Q = Q_1 \cup Q_2$ be a UCQ of self-join-free body-isomorphic acyclic CQs. If Q_1 is not free-path guarded, then $\text{ENUM}\langle Q \rangle$ is not in $\text{DelayC}_{\text{lin}}$, assuming MAT-MUL .*

PROOF. Let A and B be Boolean $n \times n$ matrices represented as binary relations, i.e., $A \subseteq \{1, \dots, n\}^2$, where $(a, b) \in A$ means that the entry in the a th row and b th column is 1. Further, let $P = (z_0, \dots, z_{k+1})$ be a free-path in Q_1 that is not guarded, and let i be the minimal index such that $z_i \notin \text{free}(Q_2)$. We assign the path variables to three roles as follows: If $0 < i < k + 1$, we define $V_a = \{z_0, \dots, z_{i-1}\}$, $V_b = \{z_i\}$ and $V_c = \{z_{i+1}, \dots, z_{k+1}\}$. Otherwise (if i is 0 or $k + 1$), we define $V_a = \{z_0\}$, $V_b = \{z_1, \dots, z_k\}$ and $V_c = \{z_{k+1}\}$. Note that, in both cases, there exists some $\alpha \in \{a, b, c\}$ such that $V_\alpha \cap \text{free}(Q_2) = \emptyset$. Intuitively, that means that some role is not guarded. Note also that since P is chordless and $k \geq 1$, there is no atom in Q that contains both a variable from V_a and a variable from V_c . Thus, we can partition the atoms into nonempty sets $\mathcal{R}_A = \{R(\vec{v}) \in \text{atoms}(Q) \mid V_c \cap \vec{v} = \emptyset\}$ and $\mathcal{R}_B = \text{atoms}(Q) \setminus \mathcal{R}_A$, and we have that the atoms of $\mathcal{R}_A$ do not contain variables of V_c , and the atoms of $\mathcal{R}_B$ do not contain variables of V_a .

Given three values (a, b, c) , we define a function $\tau_{(a,b,c)} : \text{var}(Q) \rightarrow \{a, b, c, \perp\}$ that assigns every variable with the value corresponding to its role:

$$\tau_{(a,b,c)}(v) = \begin{cases} a & \text{if } v \in V_a, \\ b & \text{if } v \in V_b, \\ c & \text{if } v \in V_c, \\ \perp & \text{otherwise,} \end{cases}$$

For a vector $\vec{v}$, we denote by $\tau_{(a,b,c)}(\vec{v})$ the vector obtained by element-wise application of $\tau_{(a,b,c)}$. We define a database instance I over Q as follows: For every atom $R(\vec{v})$, if $R(\vec{v}) \in \mathcal{R}_A$ we set $R^I = \{\tau_{(a,b,\perp)}(\vec{v}) \mid (a, b) \in A\}$, and if $R(\vec{v}) \in \mathcal{R}_B$ we set $R^I = \{\tau_{(\perp,b,c)}(\vec{v}) \mid (b, c) \in B\}$. Note that every relation is defined only once since $\mathcal{R}_A$ and $\mathcal{R}_B$ are disjoint and Q is self-join-free.

Consider an answer $\mu \in Q(I)$. In the case that $0 < i < k + 1$, we have that $\mu(z_0) = \dots = \mu(z_{i-1}) = a$, $\mu(z_i) = b$ and $\mu(z_{i+1}) = \dots = \mu(z_{k+1}) = c$ for some $(a, b) \in A$ and $(b, c) \in B$, and in case that $i \in \{0, k + 1\}$, we have that $\mu(z_0) = a$, $\mu(z_1) = \dots = \mu(z_k) = b$ and $\mu(z_{k+1}) = c$ for some $(a, b) \in A$ and $(b, c) \in B$. This is since the variables z_i are connected via the path in both CQs. In either case, $\mu(\text{free}(Q_1))$ is a tuple only containing the values a, c , and $\perp$. So the answers to Q_1 represent the answer to the matrix multiplication task we started with. We now need to verify that the answers to Q_2 do not interfere with the reduction. If $0 < i < k + 1$, $\mu(\text{free}(Q_2))$ is a tuple only containing the values $\{a, c, \perp\}$; if $i = 0$, it only contains $\{b, c, \perp\}$; and if $i = k + 1$, it only contains $\{a, b, \perp\}$. Thus, the number of answers to Q is at most of size $2n^2$. Assume by contradiction that we can enumerate the solutions of $Q(I)$ with linear preprocessing and constant delay. To distinguish the answers of Q_1 from those of Q_2 , we can concatenate the variable names to the values, as described in Lemma 16. That way, we can ignore the answers that correspond to the values (a, b) or (b, c) ,

and use the (a, c) pairs as the answers to matrix multiplication. This solves matrix multiplication in $O(n^2)$ time, in contradiction to MAT-MUL. $\square$

In Example 25, we encounter a UCQ where both CQs are free-path guarded, but Q_1 is not bypass guarded. We can encode 4-CLIQUE in every UCQ with this property.

LEMMA 28. *Let $Q = Q_1 \cup Q_2$ be a UCQ of self-join-free body-isomorphic acyclic CQs. If Q_1 and Q_2 are free-path guarded and Q_1 is not bypass guarded, then $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$, assuming 4-CLIQUE.*

PROOF. Let $G = (V, E)$ be a graph with $|V| = n$. We show how to solve the 4-CLIQUE problem on G in time $O(n^3)$ if $\text{ENUM}\langle Q \rangle$ is in $\text{DelayC}_{\text{lin}}$. Let P be a free-path in Q_1 and let $u \notin \text{free}(Q_2)$ such that u appears in two subsequent P-atoms.

We first claim that, under the conditions of this lemma, P is of length 2. Let $P = (z_0, \dots, z_{k+1})$ and $1 \leq i \leq k$ such that $\{u, z_{i-1}, z_i\}$ and $\{u, z_i, z_{i+1}\}$ are contained in edges of $\mathcal{H}(Q)$. As P is chordless, there is no edge containing $\{z_{i-1}, z_{i+1}\}$, thus the path (z_{i-1}, u, z_{i+1}) is a chordless path. As Q_1 is free-path guarded, $z_{i-1}, z_{i+1} \in \text{free}(Q_2)$ and since $u \notin \text{free}(Q_2)$, this is a free-path of Q_2 . Since Q_2 is free-path guarded, we have that $z_{i-1}, z_{i+1} \in \text{free}(Q_1)$. Since P is a free-path in Q_1 , the only variables of P that are free in Q_1 appear in the edges of the path, and so $i = k = 1$, and P is of the form (z_0, z_1, z_2) . Therefore, there exist atoms R_1 and R_2 with $\{z_0, z_1, u\} \subseteq \text{var}(R_1)$ and $\{z_1, z_2, u\} \subseteq \text{var}(R_2)$.

Given three values (a, b, c) , we define a function $\tau_{(a,b,c)} : \text{var}(Q) \rightarrow \{a, b, c\}$ as follows:

$$\tau_{a,b,c}(v) = \begin{cases} a & \text{if } v \in \{z_0, z_2\}, \\ b & \text{if } v = z_1, \\ c & \text{if } v = u, \\ \perp & \text{otherwise.} \end{cases}$$

For every atom $R(\vec{v}) \in \text{atoms}(Q)$, we define $R^I = \{\tau_{a,b,c}(\vec{v}) \mid (a, b, c) \text{ a triangle in } G\}$. Every relation is defined only once since Q is self-join-free. Note that $|R^I| \in O(n^3)$, as there are at most n^3 triangles in G , and that we can construct I with $O(n^3)$ time.

Consider an answer $\mu|_{\text{free}(Q_1)} \in Q_1(I)$. Since R_1 and R_2 are atoms in Q_1 , we are guaranteed that $(\mu(z_0), \mu(z_1), \mu(u))$ and $(\mu(z_1), \mu(z_2), \mu(u))$ form triangles in G . Therefore, the graph contains a 4-clique if and only if there is an edge $(\mu(z_0), \mu(z_2))$. As $z_0, z_2 \in \text{free}(Q_1)$, it suffices to check every $\mu|_{\text{free}(Q_1)} \in Q_1(I)$ for this property. Since Q_1 is free-path guarded, we know that z_1 is existential in Q_1 but free in Q_2 . This means that $\text{free}(Q_1) \neq \text{free}(Q_2)$ and so there is no homomorphism from Q_2 to Q_1 . We can therefore use the mappings from Lemma 16 in order to distinguish the answers of Q_1 from those of Q_2 . We have that $\{z_0, z_1, z_2, u\}$ is neither contained in $\text{free}(Q_1)$ nor in $\text{free}(Q_2)$. Thus, $|Q(I)| \in O(n^3)$. If $\text{ENUM}\langle Q \rangle$ is in $\text{DelayC}_{\text{lin}}$, we can construct the database instance, compute Q , and check every answer to Q_1 for an edge of the form $(\mu(z_0), \mu(z_2))$ in total time $O(n^3)$, which contradicts 4-CLIQUE. $\square$

In the next section, we show that Lemma 27 and Lemma 28 cover all intractable cases of the UCQs discussed in this section.

4.3 Complete Classification of Fragments of UCQs

We show next that the hardness proofs given above complete our tractability results into a dichotomy for the fragments of UCQs we examined. We first establish this for unions of two body-isomorphic acyclic CQs, and then conclude the same for unions of two intractable CQs.

To prove that any union of two body-isomorphic acyclic CQs that is not covered by Lemma 27 and Lemma 28 has a free-connex union extension, we need some observations regarding the place of appearance of relevant variables in join-trees. Recall that we call $(v_1, \dots, v_n)$ a path of variables in a query Q if, for all $1 \leq i < n$, there exists an atom $R(\vec{u}_i)$ in Q such that $\{v_i, v_{i+1}\} \subseteq \vec{v}$.

PROPOSITION 29. *Let $(v_1, \dots, v_n)$ be a path of variables in an acyclic query Q , and let A_s and A_t be atoms containing $\{v_1, v_2\}$ and $\{v_{n-1}, v_n\}$, respectively. For all $1 \leq i < n$, the simple path between A_s and A_t on a join-tree of Q contains an atom $R_i(\vec{u}_i)$ such that $\{v_i, v_{i+1}\} \subseteq \vec{u}_i$.*

PROOF. We prove this by induction on n . If $n = 3$, this trivially holds as the endpoints, A_s and A_t , contain the required variables. We now assume this proposition holds for paths of length $n - 1$ and show that it also holds for paths of length n where $n \geq 4$. Consider the simple path between A_s and a node containing $\{v_{n-2}, v_{n-1}\}$. Let A_m be the first node on that path that contains $\{v_{n-2}, v_{n-1}\}$. Denote by P_s the simple path between A_s and A_m . Note that A_m is the only node on P_s that contains $\{v_{n-2}, v_{n-1}\}$. By the induction assumption, P_s contains $\{v_i, v_{i+1}\}$ for all $1 \leq i < n - 1$, and in particular it contains $\{v_{n-3}, v_{n-2}\}$. Denote by P_t the simple path between A_m and A_t , and denote by P the concatenation of P_s and P_t . The path P goes between A_s and A_t and contains $\{v_i, v_{i+1}\}$ for all $1 \leq i < n$. We claim that P is a simple path. Assume by contradiction that P is not simple. This means that there is a node A_v that P_s and P_t share other than A_m . Due to the running intersection property, since A_v is on the simple path between A_m and A_t , A_v contains v_{n-1} . Denote by A_{m-1} the node preceding A_m on P_s . Due to the running intersection property, since A_{m-1} is on the simple path between A_v and A_m , A_{m-1} contains v_{n-1} too. Since a node containing $\{v_{n-3}, v_{n-2}\}$ is on P_s , and since A_{m-1} is on the simple path between this node and A_m , we have that A_{m-1} contains v_{n-2} . Overall, we have that A_{m-1} contains $\{v_{n-2}, v_{n-1}\}$, which is a contradiction to the fact that A_m is the only such node on P_s . $\square$

PROPOSITION 30. *Let $(v_1, \dots, v_n)$ be a path of variables in an acyclic query Q , and let A_s and A_t be atoms containing $\{v_1, v_2\}$ and $\{v_{n-1}, v_n\}$, respectively. If A_1 and A_2 are two subsequent nodes on the simple path P between A_s and A_t on a join-tree of Q , then there exists some $1 \leq i \leq n$ such that $v_i \in A_1 \cap A_2$.*

PROOF. Denote the subpath of P between A_s and A_1 by P_s , and the subpath between A_2 and A_t by P_t . Denote by i the maximal index such that $v_i \in \text{var}(P_s)$. If $i = n$, note that an atom in P_s and an atom in P_t both contain v_i . Otherwise, $i < n$. According to Proposition 29 and since all atoms of P appear in either P_s or P_t , an atom containing the variables $\{v_i, v_{i+1}\}$ must appear in P_s or P_t . It does not appear in P_s because of the maximality of i , so it appears in P_t . In this case too, an atom in P_s and an atom in P_t both contain v_i . Since A_1 and A_2 are on the path between these atoms, due to the running intersection property, $v_i \in A_1 \cap A_2$. $\square$

Using this proposition, we can prove the structural property that we need.

LEMMA 31. *Let $Q = Q_1 \cup Q_2$ be a UCQ of body-isomorphic acyclic CQs, where Q_1 and Q_2 are free-path guarded, and Q_1 is bypass guarded, and let $P = (z_0, \dots, z_{k+1})$ be a free-path in Q_1 . There exists a join-tree T for Q with a subtree T_P such that:*

- $\text{var}(P) \subseteq \text{var}(T_P)$.
- For every variable u that appears in two different atoms of T_P :
 - $u \in \text{free}(Q_2)$.
 - There is an atom $R(\vec{v})$ in T_P such that $u, z_i \in \vec{v}$ for some $0 < i < k + 1$.

PROOF. Recall that according to our notation, we assume that body-isomorphic CQs have exactly the same body. Thus, T is also a join-tree for Q_2 . In the following, we refer to the body of Q when we make statements that apply to the bodies of both Q_1 and Q_2 .

Consider a path $P_A = (A_1, \dots, A_s)$ between two atoms on a join tree. We define a *contraction step* for a path of length 2 or more: if there exists j such that $A_j \cap A_{j+1} \subseteq A_1 \cap A_s$, then remove the edge (A_j, A_{j+1}) and add the edge (A_1, A_s) . A path on a join-tree is said to be *fully contracted* if

none of its subpaths can be contracted. Given any two atoms on a join-tree, it is possible to fully contract the path between them by performing any arbitrary sequence of contraction steps until it is no longer possible: the process will end as every contraction step reduces the length of the path.

We now claim that the graph T' obtained from such a contraction step of a path P_A on a join-tree T , remains a join-tree. It is still a tree since it remains connected and with the same number of edges as before. It is left to show the running intersection property. We start with some observations regarding T . Since the running intersection property holds in T , for all $1 \leq i \leq s$, $A_1 \cap A_s \subseteq A_i$. Since $A_j \cap A_{j+1} \subseteq A_1 \cap A_s$, we also have that $A_j \cap A_{j+1} \subseteq A_i$. Now consider two nodes B and C . We need to show that every node on the simple path between them in T' contains $B \cap C$. If it is the same path as in T , then we are done. Otherwise, a path between B and C in T' can be obtained by using the simple path between them in T and replacing the edge (A_j, A_{j+1}) with $(A_{j+1}, \dots, A_s, A_1, \dots, A_j)$. The simple path between B and C is contained in this path. This means that atoms on the simple path between B and C in T' are either: (1) on the simple path between B and C in T , and therefore contain $B \cap C$; (2) on the path P_A and therefore contain $A_j \cap A_{j+1}$. Since A_j and A_{j+1} are on the path between B and C in T , we have that $B \cap C \subseteq A_j \cap A_{j+1}$, so in this case too, the atoms contain $B \cap C$. This proves that the contracted graph is indeed a join-tree. By induction, if we fully contract a path on a join-tree, we still have a join-tree.

Now let T be a join-tree of Q . We consider some path in T between an atom containing $\{z_0, z_1\}$ and an atom containing $\{z_k, z_{k+1}\}$. Take the unique subpath of it containing only one atom with $\{z_0, z_1\}$ and one atom with $\{z_k, z_{k+1}\}$, and fully contract it. We denote this fully contracted subpath as $T_P = (A_1, \dots, A_s)$. Due to Proposition 29, $\text{var}(P) \subseteq \text{var}(T_P)$.

First, we claim that every variable u that appears in two or more atoms of T_P is part of a chordless path from z_0 to z_{k+1} . We first show a chordless path from u to z_{k+1} . Denote the last atom on T_P containing u by A_i . If A_i contains z_{k+1} , we have found the chordless path (u, z_{k+1}) , and we are done. Otherwise, A_i is not the last atom on T_P . It is also not the first atom on T_P , as another atom contains u . Consider the subpath A_{i-1}, A_i, A_{i+1} . Since it is fully contracted, $A_i \cap A_{i+1} \not\subseteq A_{i-1} \cap A_{i+1}$. This means that there is a variable u_{+1} in A_i and in A_{i+1} that does not appear in A_{i-1} . Now consider the last atom containing u_{+1} , and continue with the same process iteratively until reaching z_{k+1} . This results in a chordless path $u, u_{+1}, \dots, u_{+m} = z_{k+1}$ with $m \geq 1$. Do the same symmetrically to find a chordless path $z_0 = u_{-n}, \dots, u_{-1}, u$ with $n \geq 1$. We now claim that the concatenation $P_u = (u_{-n}, \dots, u_{-1}, u, u_{+1}, \dots, u_{+m})$ is chordless. We prove that u_{-t} and $u_{+\ell}$ are not neighbors for all $1 \leq t \leq n$ and $1 \leq \ell \leq m$ by induction on $\ell + t$. Out of the atoms containing u , the variable u_{+1} only appears in the last atom by construction: if $u_{+1} = z_{k+1}$, this is true since z_{k+1} appears only in one atom, and otherwise it is true because this is how we chose u_{+1} . Similarly u_{-t} only appears in the first. Since there are at least two atoms of the path containing u , we have that u_{-1} and u_{+1} are not neighbors, and this proves the induction base. Next, assume by way of contradiction that u_{-t} and $u_{+\ell}$ are neighbors. By using the induction assumption, we have that $u_{-t}, \dots, u_{-1}, u, u_{+1}, \dots, u_{+\ell}, u_{-t}$ is a chordless cycle of length four or more, contradicting the fact that Q is acyclic and therefore chordal. This proves that u is part of the chordless path P_u from z_0 to z_{k+1} .

Assume by contradiction that a variable $u \notin \text{free}(Q_2)$ appears in two distinct atoms of T_P . There is a chordless path from z_0 to z_{k+1} that contains u . Denote this path P_u . Take a subpath of P_u starting with the last variable on P_u before u that is in $\text{free}(Q_2)$, and ending with the first variable on P_u after u that is in $\text{free}(Q_2)$. Such variables exist on this path because $z_0, z_{k+1} \in \text{free}(Q_2)$. This subpath is a free-path in Q_2 , and since Q_2 is free-path guarded, $u \in \text{free}(Q_1)$. Next consider two neighboring atoms on T_P that contain u . According to Proposition 30, there exists some z_i that appears in both atoms. Note that $i > 0$ and $i < k + 1$ since the path only contains one atom

with z_0 and one atom with z_{k+1} . Since Q_1 is bypass-guarded and $u \notin \text{free}(Q_2)$, it is not possible that there is both an atom with $\{z_{i-1}, z_i, u\}$ and an atom with $\{z_i, z_{i+1}, u\}$ in Q . Without loss of generality, assume there is no atom with $\{z_i, z_{i+1}, u\}$. Since the query is acyclic, this means that u and z_{i+1} are not neighbors (if the three variables z_i, z_{i+1}, u are pairwise neighbors in an acyclic graph, then necessarily they all appear in the same hyperedge). Then, there is a chordless path $(u, z_i, z_{i+1}, \dots, z_{k+1})$. Since $u \in \text{free}(Q_1)$, it is a free-path. This contradicts the fact that Q_1 is free-path guarded since $u \notin \text{free}(Q_2)$.

It is left to show that there is an atom $R(\vec{v})$ in T_P such that $u, z_i \in \vec{v}$ for some $0 < i < k + 1$. Since u appears in two distinct atoms of T_P , and since T_P is a join-tree, u also appears in two adjacent atoms of T_P . According to Proposition 30, there exists $0 \leq i \leq k + 1$ such that z_i is in those atoms. This cannot be 0 or $k + 1$ because they appear only in one atom in T_P . $\square$

We are now ready to show that the properties free-path guardedness and bypass guardedness imply free-connexity.

LEMMA 32. *Let $Q = Q_1 \cup Q_2$ be a UCQ of body-isomorphic acyclic CQs. If Q_1 and Q_2 are both free-path guarded and bypass guarded, then Q is free-connex.*

PROOF. We describe how to iteratively build a union extension for each CQ. In every step, we take one free-path among the queries in Q and add a virtual atom in order to eliminate this free-path. More specifically, let $P = (z_0, \dots, z_{k+1})$ be a free-path in Q_1 . Take T_P according to Lemma 31, and denote by V_P the variables of P and all variables that appear in more than one atom of T_P . We add the atom $R(V_P)$ to both Q_1 and Q_2 and obtain Q_1^+ and Q_2^+ , respectively. We will show that repeatedly applying such steps eventually leads to free-connex union extensions for Q_1 and Q_2 .

First, we claim that Q_2 supplies V_P . It is guaranteed that $V_P \subseteq \text{free}(Q_2)$ since P is guarded and due to Lemma 31. We now show that Q_2 is acyclic V_P -connex. We know that T_P is a subtree of T that contains V_P , but it may contain additional variables, each appearing in only atom, so we need to modify the tree. For every vertex A_i in T_P , add another vertex with $A'_i = \text{var}(A_i) \cap V_P$ and an edge (A_i, A'_i) . Then, for every edge (A_i, A_j) in T_P , replace it with the edge (A'_i, A'_j) . The new graph is a tree since it is connected and the number of added vertices is equal to the number of added edges (hence, after the modification, the number of edges remains equal to the number of vertices minus one). We next show that the running intersection property is maintained. For every edge (A_i, A_j) removed, $\text{var}(A_i) \cap \text{var}(A_j) \subseteq V_P$, and so by definition of the new vertices, $\text{var}(A_i) \cap \text{var}(A_j) = \text{var}(A'_i) \cap \text{var}(A'_j)$. Given two nodes B and C of the join-tree, the path between them in the modified join-tree is similar to the path in the original join-tree, except it may contain the node (A'_i, A'_j) if it contained the node (A_i, A_j) before. Since the running intersection property holds in T , every node on the path between B and C contains $\text{var}(B) \cap \text{var}(C)$. Since $\text{var}(A_i) \cap \text{var}(A_j) = \text{var}(A'_i) \cap \text{var}(A'_j)$, the running intersection property also holds in the modified tree. As the variables of the subtree that consists of the new vertices are exactly V_P , this concludes the proof that Q_2 supplies V_P .

After the extension step we described, there are no free-paths that start in z_0 and end in z_{k+1} since these variables are now neighbors. If both of the CQs are now free-connex, then we are done. Otherwise, we use the extension recursively. We can apply the extension again as we show next that the UCQ $Q_1^+ \cup Q_2^+$ conforms to the conditions of this lemma. Note that, after a free-path from z_0 to z_{k+1} is treated, and even after future extension, there will be no free-path from z_0 to z_{k+1} since they are now neighbors. Since there is a finite number of variable pairs, after a finite number of such steps, all pairs that have a free-path between them are resolved. At this point, we have an acyclic extension with no free-paths, so it is free-connex. It is left to prove that the conditions of

this lemma are maintained after the extension steps as long as at least one of the extended CQs is not free-connex. We show that in the following three claims. $\square$

CLAIM 1. Q_1^+ and Q_2^+ are body-isomorphic acyclic.

PROOF OF CLAIM 1. First recall that we assume that the CQs are given in the notation defined in the beginning of Section 4.2, so the two CQs have exactly the same body. Since we add the same atom to both CQs, the extensions also have the same body. We show a join-tree T^+ for the extension. Take the join-tree T according to Lemma 31, and add the vertex $R(V_P)$. Remove all edges in T_P and add an edge between $R(V_P)$ and every atom in T_P . This construction results in a connected graph with no cycles, and so it is a tree. We claim that the running intersection property is preserved. Let B and C be two nodes in T^+ . We first handle the case that none of B and C are $R(V_P)$. We need to show that every node on the path between them in T^+ contains $B \cap C$. Since this property holds in T , every node on the path between them in T^+ that also appears on the path between them in T preserves this property. By construction, the only new node that may appear on this path is $R(V_P)$. In this case, the two nodes before and after $R(V_P)$ on this path contain $B \cap C$. By definition, V_P contains all variables that appear in more than one atom in T_P , so $R(V_P)$ contains every variable that appears in more than one of its neighbors, and it also contains $B \cap C$. It is left to handle the second case. Assume without loss of generality that $C = R(V_P)$. We need to handle the path between $R(V_P)$ and B . Let $v \in V_P \cap B$. Since $v \in V_P$, there exists some node A_v in T_P that contains it. Consider the simple path in T between A_v and B , and let A'_v be the last node on this path which is in T_P . Due to the running intersection property, every node on this path contains $A_v \cap B$, and so it also contains v . The edge from V_P to A'_v and the simple path from A'_v to B is therefore a simple path in T^+ from V_P to B that contains v . $\square$

CLAIM 2. Q_1^+ and Q_2^+ are free-path guarded.

PROOF OF CLAIM 2. Since the original CQs are free-path guarded, every free-path in the extension that is also a free-path in the original query is guarded. According to our construction, the only atom that was added in the extension contains exactly V_P . Thus, a new free-path $(v_0, \dots, v_{m+1})$ contains $v_j, v_{j+1} \in V_P \subseteq \text{free}(Q_2) = \text{free}(Q_2^+)$. In particular, since the variables in the center of a free-path must be existential, Q_2^+ does not contain new free-paths. It is left to handle free-paths that appear in Q_1^+ but not in Q_1 .

Let $P' = (v_0, \dots, v_{m+1})$ be a free-path in Q_1^+ but not in Q_1 , and let $v_j, v_{j+1} \in V_P$. We need to show that $v_i \in \text{free}(Q_2^+)$ for all $0 \leq i \leq m+1$. First note that $v_j, v_{j+1} \in V_P \subseteq \text{free}(Q_2) = \text{free}(Q_2^+)$. We next prove the same for v_0 and v_{m+1} .

We first claim that there is a path P_{mid} between v_j and v_{j+1} that goes only through existential variables in Q_1 . We prove that every variable in V_P is either of the form z_i such that $0 < i < k+1$ (recall that these are the variables in the center of the eliminated free-path) or it has a neighbor of that form. Let $v \in V_P$. By definition of V_P , either there exists $0 \leq i \leq k+1$ such that $v = z_i$ or v appear in two atoms or more in T_P . In the first case, if $0 < i < k+1$, then it is of the required form. Otherwise, if $i = 0$, it has the neighbor z_1 , and if $i = k+1$, it has the neighbor z_k . In the second case (if v appear in two atoms or more in T_P), according to Lemma 31, v has a neighbor of the required form. Since $v_j, v_{j+1} \in V_P$, this proves that each of v_j and v_{j+1} is either of the form z_i such that $0 < i < k+1$ or it has a neighbor of that form. Since all z_i such that $0 < i < k+1$ are connected through P , there is a path P_{mid} between v_j and v_{j+1} that goes only through existential variables in Q_1 .

Since P' is chordless, no variable in P' other than v_j and v_{j+1} is in V_P . One consequence of this is that $P_s = (v_0, \dots, v_j)$ and $P_t = (v_{j+1}, \dots, v_{m+1})$ are also paths in Q_1 . Another consequence is that v_0 and v_{m+1} are not both in V_P . Since P' is chordless and $m \geq 1$, we have that v_0 and v_{m+1} are

not neighbors in Q^+ . Since they are not both on the new atom, v_0 and v_{m+1} are not neighbors in Q either. By concatenating P_s , P_{mid} , and P_t , we obtain a path in Q from v_0 to v_{m+1} that goes only through existential variables in Q_1 . Take a simple chordless path P_ℓ contained in it. Since v_0 and v_{m+1} are not neighbors, the path P_ℓ is of length two at least, and so it is a free-path in Q_1 . Since Q_1 is free-path guarded, we conclude that $v_0, v_{m+1} \in \text{free}(Q_2)$.

So far, we have that $v_0, v_j, v_{j+1}, v_{m+1} \in \text{free}(Q_2)$. Assume by contradiction that there exists $0 < i < j$ such that $v_i \notin \text{free}(Q_2)$, and consider the subpath of P_s beginning with the last variable before v_i on P_s that is in $\text{free}(Q_2)$ and ending with the first after v_i on P_s that is in $\text{free}(Q_2)$. This is a free-path in Q_2 containing v_i . We know that $v_i \notin \text{free}(Q_1)$ as it is in the center of the free-path P' , but this contradicts the fact that Q_2 is free-path guarded. In the same way, we can show that $v_i \in \text{free}(Q_2)$ for all $j + 1 < i < m + 1$. Overall, we have seen that $v_i \in \text{free}(Q_2) = \text{free}(Q_2^+)$ for all $0 \leq i \leq m + 1$ as required. $\square$

CLAIM 3. Q_1^+ and Q_2^+ are bypass guarded.

PROOF OF CLAIM 3. First, let $P' = (t_0, \dots, t_{m+1})$ be a free-path in Q_2^+ , and assume by contradiction that there exists some $u \notin \text{free}(Q_1^+)$ that appears in two subsequent P' -atoms. This means that there exists i such that Q_2^+ has an atom containing $\{t_{i-1}, t_i, u\}$ and an atom containing $\{t_i, t_{i+1}, u\}$. As explained in the previous claim, Q_2^+ has no new free-paths, so P' is a free-path in Q_2 as well. Since Q_2 is bypass guarded, u does not appear in two subsequent P' -atoms in Q_2 , so one of these atoms is new in Q_2^+ . Assume without loss of generality that it is $\{t_i, t_{i+1}, u\}$. Then, $\{t_i, t_{i+1}, u\} \subseteq V_P \subseteq \text{free}(Q_2)$. This contradicts the fact that $t_i \notin \text{free}(Q_2)$ since P' is a free-path.

Now let $P' = (t_0, \dots, t_{m+1})$ be a free-path in Q_1^+ . Assume by contradiction that there exists some $u \notin \text{free}(Q_2^+)$, that appears in two subsequent P' -atoms. This means that Q^+ has an atom containing $\{t_{i-1}, t_i, u\}$ and an atom containing $\{t_i, t_{i+1}, u\}$. Since P' is chordless, t_{i-1} and t_{i+1} are not neighbors, and so (t_{i-1}, u, t_{i+1}) is a chordless path as well. According to Claim 2, $\text{var}(P') \subseteq \text{free}(Q_2^+)$, and so (t_{i-1}, u, t_{i+1}) is a free-path in Q_2^+ . According to Claim 2 again, $\{t_{i-1}, u, t_{i+1}\} \subseteq \text{free}(Q_1^+)$. Since the only free variables on a free-path are at its ends, this means that P' is of length two, and $i = m + 1$. Since $u \notin \text{free}(Q_2)$ and $V_P \subseteq \text{free}(Q_2)$, we have that the atoms containing $\{t_0, t_1, u\}$ and $\{t_1, t_2, u\}$ are not the added atom, and so they appear also in Q_2 and in Q_1 . This means that P' is a free-path in Q_1 , and $u \notin \text{free}(Q_2)$ appears in two subsequent atoms of P' in Q_1 . Thus, Q_1 is not bypass-guarded, in contradiction to the conditions of this lemma. $\square$

This proves that the lemma can be applied iteratively.

Since Lemma 27, Lemma 28, and Lemma 32 cover all cases of a union of two self-join-free body-isomorphic acyclic CQs, we have a dichotomy that characterizes the fragment of UCQs we discuss.

THEOREM 33. Let $Q = Q_1 \cup Q_2$ be a UCQ of self-join-free body-isomorphic CQs.

- If Q_1 and Q_2 are acyclic free-path guarded and bypass guarded, then Q is free-connex and $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$.
- Otherwise, Q is not free-connex and $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$, under the assumptions *HYPER-CLIQUE*, *MAT-MUL* and *4-CLIQUE*.

PROOF. Since the CQs are body-isomorphic, they are either both acyclic or both cyclic. First assume that the CQs are acyclic. By Lemma 32, if Q_1 and Q_2 are both free-path guarded and bypass guarded, then Q is free-connex, and $\text{ENUM}\langle Q \rangle \in \text{DelayC}_{\text{lin}}$ by Theorem 14. Otherwise, either one of Q_1 and Q_2 is not free-path guarded, or they both are but one of them is not bypass guarded. In these cases, by Lemma 27 and Lemma 28, $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$ assuming *MAT-MUL* and *4-CLIQUE*. By Theorem 21, if the CQs are cyclic, $\text{ENUM}\langle Q \rangle$ is not in $\text{DelayC}_{\text{lin}}$ assuming *HYPERCLIQUE*. In all cases where $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$, we know that Q is not free-connex by Theorem 14. $\square$

By combining Theorem 33 with Theorem 21, we have the following dichotomy for the case of unions containing exactly two intractable CQs.

THEOREM 34. *Let $Q = Q_1 \cup Q_2$ be a union of difficult CQs.*

- *If Q is free-connex, then $\text{ENUM}(Q) \in \text{DelayC}_{\text{lin}}$.*
- *If Q is not free-connex, then $\text{ENUM}(Q) \notin \text{DelayC}_{\text{lin}}$, assuming MAT-MUL , HYPERCLIQUE and 4-CLIQUE .*

PROOF. If Q is free-connex, Theorem 4 proves that it is tractable. If Q_1 and Q_2 are acyclic and body-isomorphic but Q is not free-connex, then Theorem 33 proves that Q is intractable. Otherwise, Q is a union of difficult CQs that does not contain body-isomorphic acyclic CQs, and Theorem 21 proves that it is intractable. $\square$

5 TOWARDS A DICHOTOMY

In this section, we examine the next steps that are required to fully characterize which UCQs are in $\text{DelayC}_{\text{lin}}$. We pinpoint some of the difficulties that must be tackled when formulating such a dichotomy, accompanied by examples. In Section 5.1, we discuss unions containing only acyclic CQs, and, in Section 5.2, we discuss those that contain at least one cyclic CQ.

5.1 Unions of Acyclic CQs

We inspect two ways of extending the results of Section 4.2. The first such extension is to a union of two CQs that are not body-isomorphic. If there is a difficult CQ Q_1 in the union where for every other CQ Q_i in the union there is no body-homomorphism from Q_i to Q_1 , we can reduce Q_1 to the union as described in Lemma 18. Since Q_1 is difficult, the union is intractable too. In case there is a body-homomorphism to the hard queries, one might think that it is sufficient for intractability to have unguarded difficult structures similarly to the previous section. This is incorrect.

Example 35 (Unresolved UCQ with CQs that are not body-isomorphic). Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, w) &\leftarrow R_1(x, z), R_2(z, y), R_3(y, w) \text{ and} \\ Q_2(x, y, w) &\leftarrow R_1(x, t_1), R_2(t_2, y), R_3(w, t_3). \end{aligned}$$

The query Q_1 is acyclic non-free-connex, while Q_2 is free-connex. Further, Q_1 is not contained in Q_2 but there is a body-homomorphism from Q_2 to Q_1 . The variable z is part of the free-path (x, z, y) in Q_1 , but the variables t_1 and t_2 that map to it via the body-homomorphism are not free in Q_2 . If we extend the notion of guarding to non-body-isomorphic CQs in the natural way, the free-path (x, z, y) is not guarded. Nevertheless, we cannot compute matrix multiplication in $O(n^2)$ time by encoding it to the free-path (x, z, y) like before. Over such a construction, there can be n^3 many results to Q_2 as w and y are not connected in Q_2 so they can have distinct values. This is not an issue when discussing body-isomorphic CQs. We do not know whether this example is in $\text{DelayC}_{\text{lin}}$. $\square$

A future characterization of the union of CQs that are not body-isomorphic would need an even more careful approach than the one used in Section 4.2 in order to handle the case that variables mapping to the free-path are not connected via other variables mapping to the free-path.

A second way of extending Section 4.2 is to consider more than two body-isomorphic acyclic CQs. This case may be tractable or intractable, and for some queries of this form, we do not have a classification yet. Example 15 shows a tractable union of such form, while the following example shows that free-paths that share edges can be especially problematic within a union.

Example 36 (Unresolved UCQ with More than Two Body-Isomorphic CQs). Let $k \geq 4$ and consider the UCQ Q containing k body-isomorphic CQs, with an atom $R_i(x_i, z)$ for every $1 \leq i \leq k-1$. The heads are all possible combinations of $k-1$ out of the k variables of the query, $\{z, x_1, \dots, x_{k-1}\}$. In the case of $k = 4$, we have the following UCQ:

$$Q_1(x_1, x_2, x_3), Q_2(x_1, x_2, z), Q_3(x_1, x_3, z), Q_4(x_2, x_3, z) \leftarrow R_1(x_1, z), R_2(x_2, z), R_3(x_3, z).$$

The query Q_1 has free-paths (x_i, z, x_j) between all possible pairs of i and j . In this case, enumerating the solutions to the UCQ is not in $\text{DelayC}_{\text{lin}}$ assuming 4-CLIQUE: Encode each relation with all edges in the input graph, and concatenate the variable names. That is, for every edge (u, v) in the graph, add $((u, x_1), (v, z))$ to R_1 . By concatenating variable names, we can identify which solutions come from which CQ as described in Lemma 16 and ignore the answers to all CQs other than Q_1 . The answers to Q_1 give us 3 vertices that have a common neighbor. We can check in constant time if every pair of the 3 vertices are neighbors, and if so, we found a 4-clique. As there can be $O(n^3)$ solutions to the UCQ, we solve 4-clique in $O(n^3)$ time. This proves that this UCQ when $k = 4$ is intractable assuming 4-CLIQUE.

If we apply the same strategy for a general k , we get that tractability of this UCQ implies solving k -clique in time $O(n^{k-1})$. However, this does not result in a lower bound for large k values: It is assumed that one can not find a k -clique in time $n^{\frac{wk}{3}-o(1)}$, which does not contradict an $O(n^{k-1})$ algorithm. In fact, for sufficiently large k , algorithms for k -clique detection that require time $O(n^{k-1})$ do exist [15]. However, this reduction does not seem to fully capture the hardness of this query, as it encodes all relations with the same set of edges. We do not know if queries of the structure given here are hard in general, or if they become easy for larger k values, as any current approach that we know of for constant delay enumeration fails for them. $\square$

We now describe some classification we can achieve when we exclude cases like Example 36.

5.1.1 Lower Bound for Isolated Free-paths. When generalizing the notion of guarding free-paths to unions of several CQs, a free-path does not have to be guarded by a single CQ. We formalize this in the following definition.

Definition 37. Let $Q = Q_1 \cup \dots \cup Q_n$ be a union of body-isomorphic CQs, and let $P = (z_0, \dots, z_{k+1})$ be a free-path in Q_1 . We say that a set $\mathcal{U} \subseteq 2^{\text{var}(P)}$ is a *union guard* for P if:

- $\{z_0, z_{k+1}\} \in \mathcal{U}$.
- For every $\{z_a, z_c\} \subseteq u \in \mathcal{U}$ with $a+1 < c$, we have that $\{z_a, z_b, z_c\} \in \mathcal{U}$ for some $a < b < c$.
- For every $u \in \mathcal{U}$, we have $u \subseteq \text{free}(Q_i)$ for some $1 \leq i \leq n$.

We sometimes refer to the set $\{z_a, z_b, z_c\}$ with $a < b < c$ as (z_a, z_b, z_c) .

Note that, when $n = 2$, if a UCQ is free-path guarded by Definition 26, every free-path P it contains has the union guard $\mathcal{U} = \{\text{var}(P)\}$. Definition 37 allows several variable sets in the union guard, and different sets are allowed to be free in different CQs. We now show that if a UCQ contains a free-path with no union guard, then the entire union is intractable.

THEOREM 38. *Let $Q = Q_1 \cup \dots \cup Q_n$ be a UCQ of body-isomorphic acyclic CQs. If there exists a free-path in Q_1 that is not union guarded, then $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$, assuming MAT-MUL.*

PROOF. Let $P = (z_0, \dots, z_{k+1})$ be a free-path of Q_1 that is not union guarded. We know that $\{z_0, z_{k+1}\} \subseteq \text{free}(Q_1)$. Since P is not union guarded, there exist some a and c such that $a+1 < c$, and $\{z_a, z_c\} \subseteq \text{free}(Q_r)$ for some $Q_r \in Q$, but for all $Q_s \in Q$ and for all $a < b < c$, we have $\{z_a, z_b, z_c\} \notin \text{free}(Q_s)$. Note that $P' = (z_a, z_{a+1}, \dots, z_c)$ is a free-path of Q_r . We define $V_a = \{z_a\}$,

$V_b = \{z_{a+1}, \dots, z_{c-1}\}$, and $V_c = \{z_c\}$. Since P' is a free-path, there is no atom in Q that contains both z_a and z_c as variables.

The rest of the proof is similar to Lemma 27. We partition the atoms into nonempty sets $\mathcal{R}_A = \{R(\vec{v}) \in \text{atoms}(Q) \mid V_c \cap \vec{v} = \emptyset\}$ and $\mathcal{R}_B = \text{atoms}(Q) \setminus \mathcal{R}_A$, and we have that the atoms of $\mathcal{R}_A$ do not contain the variable of V_c , and the atoms of $\mathcal{R}_B$ do not contain the variable of V_a . Let A and B be Boolean $n \times n$ matrices represented as binary relations. We define a database instance I over Q as follows: For every atom $R(\vec{v})$, if $R(\vec{v}) \in \mathcal{R}_A$ we set $R^I = \{\tau_{(a,b,\perp)}(\vec{v}) \mid (a,b) \in A\}$, and if $R(\vec{v}) \in \mathcal{R}_B$ we set $R^I = \{\tau_{(\perp,b,c)}(\vec{v}) \mid (b,c) \in B\}$. Here, $\tau_{(a,b,c)}$ is the function defined in Lemma 27.

Consider an answer $\mu \in Q(I)$. Since the variables z_i are connected via the path in all CQs, we have that $\mu(z_a) = a$, $\mu(z_{a+1}) = \dots = \mu(z_{c-1}) = b$ and $\mu(z_c) = c$ for some $(a,b) \in A$ and $(b,c) \in B$. Since $\{z_a, z_c\} \subseteq \text{free}(Q_r)$, the product AB is encoded in $Q_r(I)$. Since for all $Q_s \in Q$ and for all $a < b < c$ we have $\{z_a, z_b, z_c\} \not\subseteq \text{free}(Q_s)$, every $Q_s(I)$ is of size at most $O(n^2)$. Thus, there are $O(n^2)$ answers to Q . Assume by contradiction that we can enumerate the solutions of $Q(I)$ with linear preprocessing and constant delay. To distinguish the answers of Q_r from those of the other CQs, we can concatenate the variable names to the values, as described in Lemma 16. That way, we can ignore the answers that correspond to the values (a,b) or (b,c) , and use the (a,c) pairs as the answers to matrix multiplication. This solves matrix multiplication in $O(n^2)$ time, in contradiction to MAT-MUL. $\square$

As we do not know of a classification for Example 36, we restrict the UCQs we consider to cases where the free-paths within each CQ do not share variables. Then, we manage to obtain a similar characterization to that of Section 4.2.

Definition 39. Let $Q = Q_1 \cup \dots \cup Q_n$ be a union of body-isomorphic CQs, and let P be a free-path of Q_1 . We say that P is *isolated* if the following two conditions hold:

- Q is $\text{var}(P)$ -connex and
- $\text{var}(P') \cap \text{var}(P) = \emptyset$ for all free-paths $P' \neq P$ in Q_1 .

Note that isolated is a stronger property than bypass-guarded. Given a free-path P , A bypass-guarded query can have a variable in two subsequent P -atoms as long as this variable is free in another CQ. An isolated free-path cannot have such a variable at all. If all free-paths in a union of body-isomorphic acyclic CQs are union guarded and isolated, we can show that the union is tractable. To achieve this result, we first need to show a structural property given in the following lemma:

LEMMA 40. Let $Q = Q_1 \cup \dots \cup Q_n$ be a UCQ, $P = (z_0, \dots, z_{k+1})$ a free-path of Q_1 and $\mathcal{U}$ a union guard of P . There exists some $\mathcal{U}' \subseteq \mathcal{U}$ such that:

- $\{z_0, z_{k+1}\} \subseteq v \in \mathcal{U}'$;
- for all $1 \leq i \leq m$, $\{z_{i-1}, z_i\} \subseteq v \in \mathcal{U}'$;
- there exists a join-tree T_P for $\mathcal{H} = (\text{var}(P), \mathcal{U}')$.

PROOF. We use the inductive definition of a union guard to define $\mathcal{U}'$ and T_P . Moreover, for the ease of explanations, we define T_P with a parent-child relation. By the first two points of Definition 37, $\{z_0, z_{k+1}\} \in \mathcal{U}$ and thus we have $(z_0, z_\ell, z_{k+1}) \in \mathcal{U}$ for some $1 \leq \ell \leq k$. We set (z_0, z_ℓ, z_{k+1}) as a root vertex of T_P . The second condition of Definition 37 defines the parent-child condition of T_P with at most two children per vertex: If $v = (z_a, z_b, z_c) \in V(T_P)$, then

- if $b > a + 1$, there exists some $\{z_a, z_j, z_b\} \in \mathcal{U}$ with $a < j < b$. Add one such vertex to the set of children of v .

- if $c > b + 1$, there exists some $\{z_b, z_j, z_c\} \in \mathcal{U}$ with $b < j < c$. Add one such vertex to the set of children of v .

Note that given $\mathcal{U}$, the choice of T_P might not be unique, and the vertices in T_P correspond to a subset $\mathcal{U}'$ of $\mathcal{U}$. $\square$

CLAIM 4. T_P is a join tree of $\mathcal{H}$.

PROOF OF THE CLAIM. For every $v \in V(T_P)$, denote by $T_P(v)$ the maximal subtree of T_P rooted in v . Note that for $v = (z_a, z_b, z_c)$, we have $\text{var}(T_P(v)) \subseteq \{z_a, \dots, z_c\}$ by construction.

We first prove that for every $v_p, v_c \in V(T_P)$, if v_c is a descendent of v_p and $z_i \in v_c \cap v_p$, then z_i is contained in every vertex on the path between v_p and v_c . Let $v_c \in T_P(v_p)$, and by way of contradiction let v_q be the first vertex the path not containing z_i . Then, by construction of T_P , the parent of v_q must either be of the form (z_i, z_a, z_b) , or (z_a, z_b, z_i) . In both cases, v_q is of the form (z_a, z_j, z_b) , and $\text{var}(T_P(v_q)) \subseteq \{z_a, \dots, z_b\}$. In the first case $a > i$, and in the second case $b < i$. In either case, $z_i \notin T_P(v_q)$, which is a contradiction to the fact that $z_i \in v_c$.

Let v_1, v_2 be two distinct vertices in T_P and $z_i \in v_1 \cap v_2$ for some $0 \leq i \leq k + 1$. Let v_3 be a vertex on the unique path from v_2 to v_1 . We make a case distinction by how v_1 and v_2 are connected. In case either $v_1 \in T_P(v_2)$ or $v_2 \in T_P(v_1)$, we already showed that v_3 contains z_i . So assume that $v_1 \notin T_P(v_2)$ and $v_2 \notin T_P(v_1)$. This means that there is some $v_p \in V(T_P) \setminus \{v_1, v_2\}$ with $v_1, v_2 \in T_P(v_p)$, and distinct children u_1, u_2 of v_p such that $v_1 \in T_P(u_1)$ and $v_2 \in T_P(u_2)$. For $v_p = (z_a, z_b, z_c)$, we have that $\text{var}(T_P(u_1)) \cap \text{var}(T_P(u_2)) = \{z_b\}$ by construction, and since $z_i \in v_1 \cap v_2$, it follows that $z_b = z_i$. Thus, either v_3 is on the unique path from v_p to v_1 or from v_p to v_2 . Since v_1 and v_2 are descendents of v_p , we already showed that $z_b \in v_3$. $\square$

CLAIM 5. For every $1 \leq i \leq k + 1$, the set $\{z_{i-1}, z_i\}$ is contained in a vertex of T_P .

PROOF OF THE CLAIM. We describe a path from the root to a vertex containing $\{z_{i-1}, z_i\}$. We start with the root, and at each step we consider a vertex $v = (z_a, z_b, z_c)$ such that $a \leq i - 1, i \leq c$. We have that either $i \leq b$ or $i - 1 \geq b$. Assume that $i \leq b$. If $b = a + 1$, we have that $a = i - 1$ and $b = i$, so we found the vertex we need. Otherwise, v has a child (z_a, z_t, z_b) with $a \leq i - 1, i \leq b$. We consider this child next. The case that $i - 1 \geq b$ is symmetrical. Since the tree is finite, the process will end and we will find such vertex. $\square$

This concludes the proof of the Lemma.

With this structural property at hand, we can show a union-extension for CQs in which every free-path is union guarded and isolated.

THEOREM 41. Let $Q = Q_1 \cup \dots \cup Q_n$ be a union of body-isomorphic acyclic CQs. If every free-path in Q is union guarded and isolated, then $\text{ENUM}(Q) \in \text{DelayC}_{\text{lin}}$.

PROOF. Let $Q_1 \in Q$. We use the fact that every free-path in Q_1 is union guarded and isolated to show that Q_1 is union-free-connex with respect to Q . Since this is true for every CQ in the union, Q is free-connex, and according to Theorem 14, $\text{ENUM}(Q) \in \text{DelayC}_{\text{lin}}$.

We show how to eliminate a free-path in Q_1 by adding virtual atoms. This process can be applied repeatedly to treat all free-paths in Q_1 . Let $P = (z_0, \dots, z_{k+1})$ be a free-path in Q_1 , and consider a join-tree T_P given by Lemma 40. We extend every CQ Q_j in Q to a union extension Q_j^+ as follows: For every $v \in T_P$, add the atom $R_v(\vec{v})$ to $\text{atoms}(Q)$, where R_v is a fresh relational symbol. In Claim 7, we show that these variables sets are supplied, so this is indeed a valid union extension. In Claim 8, we show that extension eliminates P without introducing new free-paths. As the proof of Claim 7 uses a bottom-up induction on T_P , we will need to use Claim 6 regarding the subtrees of T_P . We use

the same notation as in the proof of Lemma 40: For every vertex $v \in V(T_P)$, we denote by $T_P(v)$ the subtree of T_P rooted in v . Note that $V(T_P(v))$ is a subset of $\mathcal{U}'$. $\square$

CLAIM 6. *Let $v \in V(T_P)$. There exists an $\text{ext-var}(T_P(v))$ -connex tree T' for the hypergraph $\mathcal{H}' = (\text{var}(Q), E(\mathcal{H}(Q)) \cup V(T_P(v)))$.*

PROOF OF THE CLAIM. Let $v = (z_a, z_b, z_c)$. By construction of T_P , we have that $\text{var}(T_P(v)) = \{z_a, z_{a+1}, \dots, z_c\}$. Denote by R the subpath $(z_a, z_{a+1}, \dots, z_c)$ of P . It is possible to show that since P is a chordless path, for every subpath of P , we have that Q is $\text{var}(R)$ -connex. Let T be an $\text{ext-var}(R)$ -connex tree for $\mathcal{H}(Q)$, and let $T_R \subseteq T$ be a connected subtree of T with $\text{var}(T_R) = \text{var}(R)$. We construct a new tree T' by first removing the edges among vertices of T_R , adding the tree $T_P(v)$ and then reconnecting the vertices of T_R to $T_P(v)$ as follows: Let $u \in V(T_R)$. Since R is a chordless path, we have that $u = \{z_s\}$ for some $a \leq s \leq c$ or $u = \{z_s, z_{s+1}\}$ for some $a \leq s \leq c-1$. Thus, there exists some vertex $w \in T_P(v)$ with $u \subseteq w$. Chose some arbitrary $w \in T_P(v)$ with this property and add an edge (u, w) .

Since T is a tree, removing the edges of T_R results in a forest. For every $u \in V(T_R)$, the connected component of this forest that contains u does not contain any other vertices in $V(T_R)$. Thus, in every step of adding an edge, we attach a new tree to $T_P(v)$, and no such tree is attached to $T_P(v)$ more than once. Thus, T' is again a tree and acyclic. $\square$

CLAIM 7. Q_1^+ is a union-extension.

PROOF. We prove via a bottom-up induction on T_P that every vertex $v \in V(T_P)$ is supplied by some union-extension of a CQ in Q .

For the base case, note that the leaves of T_P are triples of the form (z_i, z_{i+1}, z_{i+2}) for $0 \leq i \leq k-1$. Since P is a chordless path, for every subpath of P we have that Q is $\text{var}(R)$ -connex. Therefore, Q is $\{z_i, z_{i+1}, z_{i+2}\}$ -connex. Since also (z_i, z_{i+1}, z_{i+2}) is contained in some $\text{free}(Q_j)$, all leaves are supplied.

Consider some vertex $v = (z_a, z_b, z_c)$ of T_P that is not a leaf, and assume that for every child v' of v , we have that every vertex in $T(v')$ is supplied. We need to show that v is supplied. Consider the $\text{ext-var}(T_P(v))$ -connex tree T' of the hypergraph $(\text{var}(Q), E(\mathcal{H}(Q)) \cup V(T_P(v)))$, that was constructed in the proof for Claim 6. In this tree, we replace the node v by the two nodes $v_1 = \{z_a, z_b\}$ and $v_2 = \{z_b, z_c\}$, and then add the edge (v_1, v_2) . For every edge $e = (u, v)$ that was lost when deleting v , we do the following: If u contains the variable z_c , add an edge (u, v_2) ; otherwise, add the edge (u, v_1) . Since no vertex in T' besides v contains both z_a and z_c , this is again a valid join tree. Moreover, both $\{z_a, z_b\}$ and $\{z_b, z_c\}$ are contained in vertices of $T' \setminus \{v_1, v_2\}$: If v has two children, then the children contain $\{z_a, z_b\}$ and $\{z_b, z_c\}$. Otherwise, if v has only one child node, we have that either $b = a+1$ or $c = a+1$. In both such cases, $\{z_a, z_b\}$ or $\{z_b, z_c\}$ is an edge of the path P and thus contained in a vertex in T' . Thus, we have that T' is $\text{ext-}\{z_a, z_b, z_c\}$ -connex acyclic. Let $Q_j \in Q$ such that $\{z_a, z_b, z_c\} \subseteq \text{free}(Q_j)$. Since every vertex in T' is supplied, there exists a union-extension Q_j^+ with $Q_j^+(\text{free}(Q)) \leftarrow R_{v_1}(\vec{v}_1), \dots, R_{v_N}(\vec{v}_N)$ and $\{v_1, \dots, v_N\} = V(T')$. Therefore, Q_j^+ supplies v . $\square$

CLAIM 8. *The set of free-paths in Q_1^+ equals the set of free-paths in Q_1 minus P .*

PROOF OF THE CLAIM. Since $\{z_0, z_{k+1}\}$ is contained in a vertex of T by Lemma 40, it is also contained in the variables of some added atom $R_v(\vec{v})$, thus P is not a free-path of Q_1^+ . For the sake of a contradiction, assume that there is some new free-path $P' = (z_0', \dots, z_{m+1}')$ in Q_1^+ . The only new edges in $\mathcal{H}(Q_1^+)$ are between variables in $\text{var}(P)$, thus $|\text{var}(P) \cap \text{var}(P')| \geq 2$. Let z_s and z_t be the first and last elements in P' that are also in P . Note that z_s and z_t are not neighbors in P . Replacing the path between z_s and z_t in P' with the path between z_s and z_t in P , we get the path

$P'' = (z'_0, \dots, z_s, z_{s+1}, \dots, z_{t-1}, z_t, \dots, z'_{m+1})$, which is a path in $\mathcal{H}(Q)$. This path contains a chordless sub-path P''' between z'_0 and z'_{m+1} containing at least one element in $\{z_s, z_{s+1}, \dots, z_{t-1}, z_t\}$. Thus, P''' is a free-path in Q_1 with a non-empty intersection with P , which is a contradiction to the assumption that every free-path is isolated. $\square$

If Q_1^+ is free connex, then we are done. Otherwise, by Claim 8, every free-path P' in Q_1^+ is a free-path in Q_1 and thus union guarded and isolated in Q_1 . Since the added atoms only contain variables of $\text{var}(P)$ and $\text{var}(P) \cap \text{var}(P') = \emptyset$, we have that P' is also union guarded and isolated in Q_1^+ . The tree-structure from Claim 6 over the root is a join-tree for the union extension. Since the extension consists of body-isomorphic acyclic queries, we can iteratively apply this process until Q_1 becomes free-connex.

Theorem 38 and Theorem 41 form a dichotomy for unions of body-isomorphic acyclic CQs where all free-paths are isolated: if all free-paths are guarded, the UCQ is in $\text{DelayC}_{\text{lin}}$; otherwise, it is not in $\text{DelayC}_{\text{lin}}$ assuming MAT-MUL. It is still left to handle cases like Example 36, of unions containing body-isomorphic acyclic CQs where some free-path is union guarded but not isolated. After solving this case, and regarding unions of difficult CQs, we should also explore unions containing body-isomorphic acyclic CQs but also other difficult CQs.

5.2 Unions Containing Cyclic CQs

We now discuss UCQs containing at least one cyclic query. We first mention that many of the observations we have regarding acyclic CQs also apply here. In the following examples, Q_1 is cyclic while Q_2 is free-connex. Example 42 shows that unions containing cyclic CQs may be tractable and covered by Theorem 14. Example 43 demonstrates that it is not enough to resolve the cyclic structures in CQs, but that we should also handle free-paths. Finally, Example 44 shows that, much like Example 35 in the acyclic case, even if all difficult structures are unguarded, the original reductions showing the intractability of single CQs may not work.

Example 42 (A Tractable UCQ Containing a Cyclic CQ). Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, z, w) &\leftarrow R_1(y, z, w, x), R_2(t, y, w), R_3(t, z, w), R_4(t, y, z), \\ Q_2(x, y, z, w) &\leftarrow R_1(x, z, w, v), R_2(y, x, w). \end{aligned}$$

The CQ Q_2 is free-connex, and $\{y, x, z, w\} \subseteq \text{free}(Q_2)$. Thus, Q_2 supplies V_1 to Q_1 . The body-homomorphism from Q_2 to Q_1 maps $V_2 = \{y, x, z, w\}$ to $V_1 = \{t, y, z, w\}$. Adding the virtual atom $R'(t, y, z, w)$ to Q_1 results in a free-connex union-extension, so $\text{ENUM}\langle Q_1 \cup Q_2 \rangle \in \text{DelayC}_{\text{lin}}$ according to Theorem 14. $\square$

Example 43 (An Intractable UCQ with Guarded Cycles). Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, v) &\leftarrow R_1(v, z, x), R_2(y, v), R_3(z, y) \text{ and} \\ Q_2(x, y, v) &\leftarrow R_1(y, v, z), R_2(x, y). \end{aligned}$$

The union $Q_1 \cup Q_2$ is intractable despite the fact that Q_2 guards the cycle variables $\{v, y, z\}$. This is due to the unguarded free-path (x, z, y) in Q_1 . Similarly to Example 23, we can encode matrix multiplication to x, z, y and conclude that $\text{ENUM}\langle Q \rangle \notin \text{DelayC}_{\text{lin}}$ assuming MAT-MUL. $\square$

Example 44 (An Unresolved UCQ with Unguarded Difficult Structures). Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, z, y, v) &\leftarrow R_1(x, z, v), R_2(z, y, v), R_3(y, x, v) \text{ and} \\ Q_2(x, z, y, v) &\leftarrow R_1(x, z, v), R_2(y, t_1, v), R_3(t_2, x, v). \end{aligned}$$

We do not know the complexity of this example. As Q_2 does not have a free variable that maps to y via a homomorphism, Q_1 is not union-free-connex, so we cannot conclude using Theorem 14

that $Q_1 \cup Q_2$ is tractable. The only difficult structure in Q_1 is the cycle x, y, z , but encoding the triangle-finding problem to this cycle in Q_1 , like we did in Example 22, could result in n^3 answers to Q_2 . This means that, if the input graph has triangles, we are not guaranteed to find one in $O(n^2)$ time by evaluating the union efficiently. $\square$

In addition to these issues, in the cyclic case, even if the original difficult structures are resolved, the extension may introduce new ones. Resolving the following example in general is left for future work.

Example 45 (An Unresolved UCQ with Guarded Difficult Structures). We start with a specific UCQ that we later generalize into a family of UCQs. Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x_2, x_3, x_4) &\leftarrow R_1(x_2, x_3, x_4), R_2(x_1, x_3, x_4), R_3(x_1, x_2, x_4), \\ Q_2(x_2, x_3, x_4) &\leftarrow R_1(x_2, x_3, x_1), R_2(x_4, x_3, v). \end{aligned}$$

There is a body-homomorphism from Q_2 to Q_1 , but Q_1 is not contained in Q_2 . Q_1 is cyclic, as it has the cycle (x_1, x_2, x_3) . This is the only difficult structure in Q_1 , i.e., $\mathcal{H}(Q_1)$ does not contain a hyperclique or a free-path. The query Q_2 is both free-connex and $\{x_2, x_3, x_4\}$ -connex, and so it supplies $\{x_2, x_3, x_4\}$. These variables are mapped to $\{x_1, x_2, x_3\}$ via the body-homomorphism to Q_1 . Nevertheless, extending Q_1 with a virtual atom $R(x_1, x_2, x_3)$ does not result in a free-connex CQ. Even though the extension “removes” all difficult structures from Q_1 , it is intractable as it introduces a new difficult structure, namely the hyperclique $\{x_1, x_2, x_3, x_4\}$.

In this case, we have $\text{ENUM}(Q_1 \cup Q_2) \notin \text{DelayC}_{\text{lin}}$ assuming 4-CLIQUE, with a reduction similar to that of Example 25: Given an input graph, compute all triangles and encode them to the three relations. For every triangle $\{a, b, c\}$, add the tuple $((a, x_2), (b, x_3), (c, x_4))$ to R_1 , $((a, x_1), (b, x_3), (c, x_4))$ to R_2 and $((a, x_1), (b, x_2), (c, x_4))$ to R_3 . By concatenating variable names, we can identify which solutions correspond to which CQ as described in Lemma 16, and thus are able to ignore the answers to Q_2 . The answers to Q_1 represent *three* vertices that appear in a 4-clique: For every answer $((b, x_2), (c, x_3), (d, x_4))$ to Q_1 , we know that $((b, x_2), (c, x_3), (d, x_4)) \in R_1$, and there exists some a with $((a, x_1), (c, x_3), (d, x_4)) \in R_2$ and $((a, x_1), (b, x_2), (d, x_4)) \in R_3$. This means that $\{a, b, c, d\}$ is a 4-clique. In the opposite direction, for every 4-clique $\{a, b, c, d\}$, we have that $\{b, c, d\}$, $\{a, c, d\}$, and $\{a, b, d\}$ are triangles. By construction, the tuple $((b, x_2), (c, x_3), (d, x_4))$ is an answer to Q_1 . As there are $O(n^3)$ triangles and there can be at most $O(n^3)$ solutions to Q_2 , we solve 4-clique in $O(n^3)$ time. This proves that the UCQ is intractable assuming 4-CLIQUE.

This example can be generalized to higher orders. There, we do not have a similar lower bound. Consider the union of the following:

$$\begin{aligned} Q_1(x_2, \dots, x_k) &\leftarrow \{R_i(x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_k) \mid 1 \leq i \leq k-1\} \\ Q_2(x_2, \dots, x_k) &\leftarrow R_1(x_2, \dots, x_{k-1}, x_1), R_2(x_k, x_3, \dots, x_{k-1}, v). \end{aligned}$$

Again, the query Q_1 is cyclic and Q_2 is free-connex. Even though Q_2 supplies variables that map to $\{x_1, \dots, x_{k-1}\}$ via a body-homomorphism, adding a virtual atom with these variables does not result in a free-connex extension, as this extension is again cyclic. As in Example 36, we can encode k -clique to this example in general, but this does not imply a lower bound for large k values. $\square$

6 MODIFIED SETTINGS

We devote this section to discussing the implications of this work when the settings are slightly different than the ones assumed so far: in Section 6.1, we discuss the common case that there are cardinality dependencies in the database schema; in Section 6.2, we discuss unions of conjunctive queries with disequalities; and in Section 6.3, we discuss the case where we want to restrict the space requirements of our algorithm. Reaching complete results regarding the tractability of UCQs

in these settings is beyond the scope of this work, but we do discuss some insights we can conclude regarding these settings following our work.

6.1 Cardinality Dependencies

In this work, we inspect the complexity of answering UCQs when we cannot assume anything on the database instance given as input. Nevertheless, usually we do know what the database we have at hand represents, and this implies restrictions on the combinations of values that the relations may hold. One way of formalizing such restrictions is through **Cardinality Dependencies (CDs)**, also known as *degree bounds* [1], given as part of the schema. Cardinality dependencies are of the form $(R : A \rightarrow B, c)$ where $A, B \subseteq \{1, \dots, \text{arity}(R)\}$ and c is a positive natural number. This dependency means that, for every database instance of this schema, the number of tuples in R with the same values in the A indices but distinct values in the B indices is at most c . **Functional Dependencies (FDs)** are a special case of cardinality dependencies where $c = 1$, and keys are a special case of FDs where $A \cup B = \{1, \dots, \text{arity}(R)\}$.

As we know, the tractable CQs over general instances are the ones with a free-connex structure. In the presence of cardinality dependencies, additional CQs are tractable. The tractable CQs can be identified by treating the dependencies as between variables and “chasing” the dependencies: the variables of the right-hand side of a dependency are added to any atom that contains the variables of its left-hand side; this extension is applied to the head of the CQ in addition to the atoms. Applying any such number of extension steps generates a *CD-extended CQ*, and if this CQ is free-connex, the original CQ is tractable [11]. This resembles the technique used in this article to show the tractability of a UCQ where we extend the CQs of the union with additional atoms to reach a tractable (free-connex) structure. Following is the formal definition of CD-Extended CQs.

Definition 46 (CD-Extended CQ [11]). Let $Q(\vec{p}) \leftarrow R_1(\vec{v}_1), \dots, R_m(\vec{v}_m)$ be a CQ over a schema with the dependencies Δ . There are two possible types of extension steps:

- The extension of an atom $R_i(\vec{v}_i)$ according to an CD $R_j : A \rightarrow b$.
Prerequisites: $\vec{v}_j[A] \subseteq \vec{v}_i$ and $\vec{v}_j[b] \notin \vec{v}_i$.
Effect: The arity of R_i increases by one, $R_i(\vec{v}_i)$ is replaced by $R_i(\vec{v}_i, \vec{v}_j[b])$. In addition, every $R_k(\vec{v}_k)$ such that $R_k = R_i$ and $k \neq i$ is replaced with $R_k(\vec{v}_k, t_k)$, where t_k is a fresh variable in every such step.
- The extension of the head $Q(\vec{p})$ according to an CD $R_j : A \rightarrow b$.
Prerequisites: $\vec{v}_j[A] \subseteq \vec{p}$ and $\vec{v}_j[b] \notin \vec{p}$.
Effect: The head is replaced by $Q(\vec{p}, \vec{v}_j[b])$.

The *CD-extension* of Q is the CQ obtained by performing all possible extension steps on Q according to CDs of Δ until a fixpoint is reached.

We devote this section to discussing whether there is a need to combine these two techniques and how this can be done. First, we discuss the need to combine the two techniques.

Example 47. Consider the UCQ $Q = Q_1 \cup Q_2$ with:

$$\begin{aligned} Q_1(x, w, u) &\leftarrow R_1(x, y), R_2(y, z), R_3(z, w), R_4(u) \\ Q_2(x, w, u) &\leftarrow R_1(x, w), R_2(t, u) \end{aligned}$$

over a schema with the FD $R_3 : 2 \rightarrow 1$.

The CQ Q_1 in itself is not free-connex, as it contains the free-path (x, y, z, w) . Even if we take the FD into account, Q_1 is not tractable. The FD-extension of Q_1 is obtained by “chasing” the FD

$w \rightarrow z$, and adding z to the query head

$$Q_1^+(x, w, u, z) \leftarrow R_1(x, y), R_2(y, z), R_3(z, w), R_4(u).$$

Q_1^+ is not free-connex as it contains the free-path (x, y, z) . This means that, by using the FD techniques alone, Q does not become tractable. If we take the union into account but ignore the FD, Q_2 can supply $\{x, w, u\}$, which map to $\{x, y, z\}$ in Q_1 via a body-homomorphism, but adding atoms with any subset of these variables cannot make Q_1 tractable, because it cannot prevent the appearance of a free-path between x and w . It can only shorten it from (x, y, z, w) to (x, z, w) . This means that by using the union techniques alone, Q does not become tractable. Nonetheless, if we combine the extensions, we both add z to the head and add an atom with $\{x, y, z\}$, we get the following tractable extension:

$$Q_1^{++}(x, w, u, z) \leftarrow R_1(x, y), R_2(y, z), R_3(z, w), R_4(u), R'(x, y, z).$$

Practically, this means that Q can be solved efficiently by first solving Q_2 , then using its results as another atom R' , then solving Q_1^{++} with the help of R' , and translating every answer to Q_1^{++} to an answer to Q_1 by projecting out z . The FD guarantees that the translation phase does not result in duplicates. If we had a cardinality dependency instead of an FD, we would have a constant number of duplicates per answer to Q_1 , and the duplicates could be ignored efficiently using the Cheater's Lemma (Lemma 7).

Example 47 shows that if we have UCQs over a schema with CDs, we can identify additional tractable cases by combining the two techniques. The two types of extensions can easily be combined in any order. To show this, we just need to modify the definitions of a free-connex union to account for CDs, and use the same proof as before.

Definition 48. Let $Q = Q_1 \cup \dots \cup Q_n$ be a UCQ over a schema with cardinality dependencies.

- A *CD-extension* for Q is of the form $Q = Q'_1 \cup \dots \cup Q'_n$ such that for all $i = 1, \dots, n$ either $Q'_i = Q_i$ or Q'_i is the CD-extension of Q_i .
- A *CD-union extension sequence* for Q is a sequence $Q^1, \dots, Q^N$ where $Q = Q^1$ and for all $1 < j \leq N$, it holds that Q^j is either a union extension or a CD-extension of Q^{j-1} .

Note that a CD-extension of a CQ may result in an increase in the arity of the head. Therefore, a CD-extension of a UCQ may not be a valid UCQ in terms of the standard definitions, as these require all CQ heads to have the same arity. However, as the algorithms we use in this article for the evaluation of UCQs do not rely on the size of the heads, we can evaluate such a union as usual.

THEOREM 49. Let Q be a UCQ over a schema with cardinality dependencies. If Q has a CD-union extension sequence ending in a UCQ containing only free-connex CQ, then $\text{ENUM}(Q) \in \text{DelayC}_{\text{lin}}$.

PROOF. Consider an extension sequence $Q^1, \dots, Q^N$. First, we inductively build an instance for Q^N . The instance for Q^1 is given as input. If Q^{i+1} is a union-extension of Q^i , use the construction given in this article for CQs without CDs (i.e., follow the provision phase described in Theorem 14) to obtain the instance for Q^{i+1} . Otherwise, Q^{i+1} is a CD-extension of Q^i , and so use the construction for CD-extensions [11, Theorem 2 (Claim 1)] instead. Then, evaluate every CQ in Q^N using the CDY algorithm. For every answer produced during the construction or in the final evaluation, translate it back to an answer to Q^1 by projecting out any extra variables (these were possibly introduced due to CD-extensions). The correctness follows from the correctness of the individual extensions. The desired time complexity is achieved using the Cheater's Lemma: since N (the length of the extension sequence) is constant with respect to the input database, the number of times every answer is produced is bound by a constant. $\square$

Note that, in Example 47, the order of applying the extension does not matter. That is, first adding R' and then adding z wherever w appears results in the same query as first adding z and then adding R' . This is not always the case. Consider the same UCQ as before but with the FD $R_3 : 1 \rightarrow 2$ instead of the reverse FD. When we treat the FD as between variables in Q_1 , we get $z \rightarrow w$, and we can add w wherever z appears in Q_1 as an FD-extension. The individual extensions are still not enough: adding w to R_2 results in the free-path (x, y, w) , and adding an atom with $\{x, y, z\}$ or a subset of these variables does not help as before. The combination of the two extensions is enough if we first use the union and only then use the FD. We get the following free-connex CQ:

$$Q'_1(x, w, u) \leftarrow R_1(x, y), R_2(y, z, w), R_3(z, w), R_4(u), R'(x, y, z, w).$$

If we apply the extensions in the opposite order, we do not have w in R' , and we have the free-path (x, z, w) , so the extension is not tractable.

Overall, we saw that the two techniques can and should be combined to find more tractable cases. However, one should be aware that it is not enough to first take the CD-extensions of all CQs in the union and then perform union-extensions. The order in which the different extensions are combined matters.

6.2 Disequalities

In this section, we discuss the enumeration complexity of unions of CQs with disequalities. In the case of individual CQs with disequalities, the disequalities have no effect on the enumeration complexity [4]. That is, one can simply ignore the disequalities, and the remaining CQ is free-connex if and only if the query is tractable (under the same assumptions as in Theorem 3). A natural question is whether this happens also with UCQs. We next show that it does not, and some tractable UCQs become intractable with the addition of disequalitites.

Example 50. Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, z, w) &\leftarrow R_1(x, y), R_2(y, z), R_3(z, z), R_4(x, z), R_5(w) \\ Q_2(x, y, z, w) &\leftarrow R_1(x, y), R_3(w, z), z \neq w. \end{aligned}$$

We can show this union is hard by encoding triangle detection to the cycle in Q_1 as usual. Due to the disequality, Q_2 has no answers over this construction, so finding an answer to the UCQ in linear time detects a triangle in linear time. If there was no disequality, Q_2 would supply variables that map to the cycle in Q_1 via a body-homomorphism, and the UCQ would become tractable.

This example shows that (unlike with CQs) the positive results presented in this article for UCQs do not hold when we ignore disequalities. Nevertheless, a simple adjustment of the proofs of Section 3 reveals cases where a UCQ with disequalities containing hard CQs is easy.

Definition 51. Let Q_1, Q_2 be CQs with disequalities.

- A *body-homomorphism* from Q_2 to Q_1 is a mapping $h : \text{var}(Q_2) \rightarrow \text{var}(Q_1)$ such that:
 - for every atom $R(\vec{v})$ of Q_2 , $R(h(\vec{v})) \in Q_1$.
 - for every disequality $v \neq u$ of Q_2 , $h(v) \neq h(u)$ is in Q_1 .
- We say that Q_2 *supplies* a set V of variables if Q_2 is free-connex and $V \subseteq \text{free}(Q_2)$.

Note that the definition of a body-homomorphism now takes the disequalities into account. Note also that the definition of supplying variables here is simpler than before as we require the

supplying CQ to be free-connex.³ We set the definition of a union extension as in Section 3 (but using the new definition of supplied variables and body-homomorphisms). Using these definitions, we can get a result similar to that of Theorem 14. We first prove the equivalent of Lemma 10 in our settings.

LEMMA 52. *Let Q_2 be a CQ with disequalities that supplies the variables $\vec{v}_2$. Given an instance I , one can compute with linear time preprocessing and constant delay $M = Q_2(I)$, and M can be translated in time $O(|M|)$ to a relation R^M such that: for every CQ Q_1 with a body-homomorphism h from Q_2 to Q_1 and for every answer $\mu_1 \in \text{full}(Q_1)(I)$, there is the tuple $\mu_1(h(\vec{v}_2)) \in R^M$.*

PROOF. Let Q_2 be a CQ with disequalities that supplies the variables $\vec{v}_2$. Since Q_2 is free-connex, it can be answered with linear preprocessing and constant delay [4]. We define $R^M = \{\mu_2(\vec{v}_2) \mid \mu_2 \in Q_2(I)\}$. Let Q_1 be a CQ such that there is a body-homomorphism h from Q_2 to Q_1 , and let $\mu_1 \in \text{full}(Q_1)(I)$. Since h is a body-homomorphism, for every atom $R(\vec{v})$ in Q_2 , $R(h(\vec{v}))$ is an atom in Q_1 , and for every disequality $v \neq u$ of Q_2 , $h(v) \neq h(u)$ is in Q_1 . Since μ_1 is an answer to $\text{full}(Q_1)$, for such atoms and disequalities $\mu_1(h(\vec{v})) \in R^I$ and $\mu_1(h(v)) \neq \mu_1(h(u))$. This means that $\mu_1 \circ h|_{\text{free}(Q_2)}$ is an answer to Q_2 , so there exists $\mu_2 \in Q_2(I)$ such that $\mu_1 \circ h|_{\text{free}(Q_2)} = \mu_2$. By construction, $\mu_1(h(\vec{v}_2)) = \mu_2(\vec{v}_2) \in R^M$. $\square$

With Lemma 52 at hand, we can apply the proof of Theorem 14 as is. We only need to use Lemma 52 instead of Lemma 10. This proves the following result:

THEOREM 53. *Let Q be a union of CQ with disequalities. If Q is free-connex, then $\text{ENUM}(Q)$ is in $\text{DelayC}_{\text{lin}}$.*

Example 54. Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, w) &\leftarrow R_1(x, z), R_2(z, y), R_3(y, w), x \neq z \text{ and} \\ Q_2(x, y, w) &\leftarrow R_1(x, y), R_2(y, w), x \neq y. \end{aligned}$$

This is the same as Example 2 except for the added disequalities. Without the disequalities, this union is easy as Q_2 supplies $\{x, y, w\}$, and adding a virtual atom with a union extension to Q_1 results in a free-connex form. Since Q_2 is free-connex and the disequality of Q_2 maps to that of Q_1 using the body-homomorphism, this union is easy even with the disequalities according to Theorem 53.

Theorem 53 can be used also to show the tractability of UCQs that are not naturally of that form. For example, in the case of the last example, the union is easy even without the disequality in Q_1 .

Example 55. Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, w) &\leftarrow R_1(x, z), R_2(z, y), R_3(y, w) \text{ and} \\ Q_2(x, y, w) &\leftarrow R_1(x, y), R_2(y, w), x \neq y. \end{aligned}$$

We can partition the answers to Q_1 to those that assign the same values to x and z and those that do not. Q is equivalent to $Q' = Q_1^\# \cup Q_1^- \cup Q_2$ with

$$\begin{aligned} Q_1^\#(x, y, w) &\leftarrow R_1(x, z), R_2(z, y), R_3(y, w), x \neq z \text{ and} \\ Q_1^-(x, y, w) &\leftarrow R_1(x, x), R_2(x, y), R_3(y, w) \text{ and} \\ Q_2(x, y, w) &\leftarrow R_1(x, y), R_2(y, w), x \neq y. \end{aligned}$$

³This is a restriction we impose for simplicity, and it is possible that a more refined condition may lead to additional tractable cases. However, this seems to require going into the details of the algorithm for CQs with disequalities, and it is left for future work.

Here, Q_2 and Q_1^- are free-connex, and $Q_1^\#$ has a free-connex union-extension, as explained in Example 54.

Unlike in the case of individual CQs, the complexity of answering unions of CQs changes with the addition of disequalities. We proved the tractability of some unions with disequalities that can be easily concluded from the results of this article, but identifying all of the tractable cases is left for future work.

6.3 Restricted Space

In this article, we only considered time bounds. The class $CD \circ Lin$ describes the problems that can be solved with the same time bounds, but with the additional restriction that the available space for writing during the enumeration phase is constant. Evaluating free-connex CQs is in $CD \circ Lin$, and Kazana offers a comparison between $DelayC_{lin}$ and $CD \circ Lin$ [20, Section 8.1.2]. All lower bounds presented in this article naturally hold for $CD \circ Lin$ as we know that, by definition of the classes, $CD \circ Lin \subseteq DelayC_{lin}$. The tractability of unions containing only free-connex CQs, as explained in Theorem 4, also holds for this class. However, in our techniques for the tractable UCQs that do not contain only tractable CQs, the memory to which we write during the enumeration phase may increase in size by a constant with every new answer. An interesting question is whether we can achieve the same time bounds when restricting the memory according to $CD \circ Lin$.

The first reason we use polynomial space is to regularize the delay and avoid duplicates. This is achieved in the Cheater's Lemma (Lemma 7) by storing all results we produce. We should mention in that context that regularizing the delay is arguably not of practical importance. In practice, once we have an answer, we will probably want to use it immediately. For this reason, one might be satisfied with simply using a relaxed complexity measure which captures the fact that an algorithm performs "just as good" as linear preprocessing and constant delay when given enough space: linear partial time (see Definition 5). The Cheater's Lemma shows that, whenever we have an algorithm $\mathcal{A}$ that runs in linear partial time, there is a linear preprocessing and constant delay algorithm $\mathcal{A}'$ that uses additional space and computes the same results, where every result in $\mathcal{A}$ is returned no later than the same result in $\mathcal{A}'$. In fact, if we denote by $Lin_{partial}$ the class of enumeration problems that can be solved by a linear partial time algorithm, $Lin_{partial} = DelayC_{lin}$. Please note that this does not imply $Lin_{partial} = CD \circ Lin$, as we currently do not know whether $DelayC_{lin} = CD \circ Lin$.

The other reason that we use large amounts of space is that we need to store the generated relations that correspond to the virtual atoms in order to evaluate the union extension as specified in Theorem 14. The discussion above implies that it makes sense to focus on eliminating this fundamental reason. As the next example demonstrates, this seems to require a different approach than the one presented in this article.

Example 56. Let $Q = Q_1 \cup Q_2$ with

$$\begin{aligned} Q_1(x, y, z, w) &\leftarrow R_1(x, y), R_2(y, z), R_3(z, x), R_4(x, w) \text{ and} \\ Q_2(x, y, z, w) &\leftarrow R_1(x, t_1), R_2(y, t_2), R_3(z, t_3), R_4(w, t_4). \end{aligned}$$

Q_1 is cyclic, so it is not possible to find an answer to it in linear time. However, since Q_2 is free-connex and supplies variables that map to the cycle $\{x, y, z\}$ in Q_1 via a body-homomorphism, and since Q_1 becomes free connex when adding an atom with these variables, the union is solvable in linear preprocessing time and constant delay by saving the results of Q_2 as a relation as suggested in Section 3. In this case, there is also a way of solving it with no additional space: use CDY to solve Q_2 efficiently; for every answer (a, b, c, d) to Q_2 , check if $a = d$, $(a, b) \in R_1$, $(b, c) \in R_2$ and $(c, a) \in R_3$; if all these conditions are met, go over all tuples $(a, e) \in R_4$ and output (a, b, c, e) . These

checks can be done in constant time per answer in the RAM model by constructing lookup tables for each relation during preprocessing. This finds all answers to $Q_1 \cup Q_2$ with constant time per answer. However, if an answer belongs to both CQs, we print it twice. The duplicates can easily be avoided by replacing R_4 in Q_1 with a modified version, lacking the tuples that cause duplicates. During preprocessing, compute $R'_4 = R_4 \setminus \{(a, e) \in R_4 \mid \exists t \text{ s.t. } (e, t) \in R_4\}$. During enumeration, for every qualifying answer (a, b, c, d) to Q_2 , go over all tuples $(a, e) \in R'_4$ and output (a, b, c, e) .

Example 56 shows that reducing the space requirements is sometimes possible. We do not know, however, whether this is the case for all free-connex UCQs. Consider Example 2. The same approach would mean that, for every answer (a, b, c) to Q_2 , we go over all tuples $(c, d) \in R_3$ and print (a, c, d) . The problem here is that we will print many duplicates: every answer (a, c, d) will be produced as many times as there are different b values with $(a, b, c) \in Q_2(I)$.

In conclusion, UCQs that contain intractable CQs can sometimes be made easy by using virtual atoms. In this article, we proposed instantiating a relation that corresponds to each virtual atom during enumeration. This instantiation can sometimes be avoided, but it requires a different approach, and it is left for future work to find when this can be done. The discussion above also gives rise to a possible relaxation that can be considered a first step: Can we evaluate such UCQs efficiently with only constant writing memory available during enumeration, when we allow for linear partial time and a constant number of duplicates per answer? Note that, in Example 56, we do not need this relaxation, so another interesting question is whether this relaxation is useful for our task or can all UCQs that can be answered with this relaxed measure also be answered with linear preprocessing and constant delay without extra space.

7 CONCLUSIONS

In this article, we study the enumeration complexity of UCQs with respect to $\text{DelayC}_{\text{lin}}$. We formalized how CQs within a union can make each other easier by supplying variables, and we introduced union extensions. Then, we defined free-connex UCQs, and showed that these are tractable. In particular, we demonstrated that UCQs containing only intractable CQs may be tractable.

We showed that in case of a union of two difficult CQs or two acyclic body-isomorphic CQs, free-connexity fully captures the tractable cases. Nevertheless, achieving a full classification of UCQs remains an open problem. Section 5 discusses the next steps that are required to fully characterize which UCQs are in $\text{DelayC}_{\text{lin}}$ and provides examples with unknown complexity. Resolving Examples 35, 36, 44, and 45 are necessary steps on the way to a future dichotomy.

In addition to the general settings, we also showed that in three other variations: cardinality dependencies in the schema, UCQs with disequalities and restricted space, we can identify tractable UCQs that contain intractable CQs. Section 6 offers a preliminary idea on how the results of the article can be applied in these variations, and leaves substantial room for future exploration of this topic. For example, Theorem 53 in Section 6.2 shows the tractability of free-connex UCQs with disequalities, but as Example 55 demonstrates, additional such queries are tractable. Can we find a characterization that captures all such tractable UCQs with disequalities?

Most of this article ignores space consumption in order to concentrate on time. An interesting question is how much space is required to achieve the time bounds described here. As Example 56 shows, sometimes we can solve UCQs with only constant memory available for writing during the enumeration phase. It is left open whether all free-connex UCQs can be solved under this restriction, and in general whether $\text{DelayC}_{\text{lin}} = \text{CD} \circ \text{Lin}$ (see the discussion in Section 6.3).

ACKNOWLEDGMENTS

The authors are greatly thankful to Benny Kimelfeld for his advice.

REFERENCES

- [1] Mahmoud Abo Khamis, Hung Q. Ngo, and Dan Suciu. 2017. What do Shannon-type inequalities, submodular width, and disjunctive datalog have to do with one another? In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 429–444.
- [2] J. Alman and V. V. Williams. 2018. Limits on all known (and some unknown) approaches to matrix multiplication. In *Proceedings of the 2018 IEEE 59th Annual Symposium on Foundations of Computer Science (FOCS)*. 580–591.
- [3] N. Alon, R. Yuster, and U. Zwick. 1997. Finding and counting given length cycles. *Algorithmica* 17, 3 (1997), 209–223. DOI: <https://doi.org/10.1007/BF02523189>
- [4] Guillaume Bagan, Arnaud Durand, and Etienne Grandjean. 2007. On acyclic conjunctive queries and constant delay enumeration. In *Proceedings of the International Workshop on Computer Science Logic*. Springer, 208–222.
- [5] Catriel Beeri, Ronald Fagin, David Maier, and Mihalis Yannakakis. 1983. On the desirability of acyclic database schemes. *Journal of the ACM (JACM)* 30, 3 (1983), 479–513.
- [6] Christoph Berkholz, Fabian Gerhardt, and Nicole Schweikardt. 2020. Constant delay enumeration for conjunctive queries: A tutorial. *ACM SIGLOG News* 7, 1 (2020), 4–33.
- [7] Christoph Berkholz, Jens Keppeler, and Nicole Schweikardt. 2017. Answering conjunctive queries under updates. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS 2017)*, (Chicago, IL, May 14–19, 2017). 303–318.
- [8] Christoph Berkholz, Jens Keppeler, and Nicole Schweikardt. 2018. Answering UCQs under updates and in the presence of integrity constraints. In *Proceedings of the 21st International Conference on Database Theory (ICDT 2018)*, (March 26–29, 2018, Vienna, Austria). 8:1–8:19.
- [9] Johann Brault-Baron. 2013. *De la pertinence de là énumération: Complexité en logiques propositionnelle et du premier ordre*. Ph.D. Dissertation. Université de Caen.
- [10] Florent Capelli and Yann Strozecki. 2018. Incremental delay enumeration: Space and time. *Discrete Applied Mathematics* (2018). DOI: <https://doi.org/10.1016/j.dam.2018.06.038>
- [11] Nofar Carmeli and Markus Kröll. 2019. Enumeration complexity of conjunctive queries with functional dependencies. *Theory of Computing Systems* (2019). DOI: <https://doi.org/10.1007/s00224-019-09937-9>
- [12] Nofar Carmeli and Markus Kröll. 2019. On the enumeration complexity of unions of conjunctive queries. In *Proceedings of the 38th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS 2019)*, Amsterdam, The Netherlands, June 30 - July 5, 2019, Dan Suciu, Sebastian Skritek, and Christoph Koch (Eds.). ACM, 134–148. DOI: <https://doi.org/10.1145/3294052.3319700>
- [13] Ashok K. Chandra and Philip M. Merlin. 1977. Optimal implementation of conjunctive queries in relational data bases. In *Proceedings of the 9th Annual ACM Symposium on Theory of Computing (STOC'77)*. Association for Computing Machinery, New York, 77–90. DOI: <https://doi.org/10.1145/800105.803397>
- [14] Arnaud Durand and Etienne Grandjean. 2007. First-order queries on structures of bounded degree are computable with constant delay. *ACM Trans. Comput. Logic* 8, 4, Article 21 (Aug. 2007). DOI: <https://doi.org/10.1145/1276920.1276923>
- [15] Friedrich Eisenbrand and Fabrizio Grandoni. 2004. On the complexity of fixed parameter clique and dominating set. *Theoretical Computer Science* 326, 1–3 (2004), 57–67.
- [16] Fernando Florenzano, Cristian Riveros, Martín Ugarte, Stijn Vansummeren, and Domagoj Vrgoc. 2018. Constant delay algorithms for regular document spanners. In *Proceedings of PODS*. ACM, 165–177.
- [17] Jörg Flum, Markus Frick, and Martin Grohe. 2002. Query evaluation via tree-decompositions. *J. ACM* 49, 6 (2002), 716–752.
- [18] Etienne Grandjean. 1996. Sorting, linear time and the satisfiability problem. *Ann. Math. Artif. Intell.* 16 (1996), 183–236. DOI: <https://doi.org/10.1007/BF02127798>
- [19] Muhammad Idris, Martin Ugarte, and Stijn Vansummeren. 2017. The dynamic Yannakakis algorithm: Compact and efficient query processing under updates. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD'17)*. ACM, New York., 1259–1274. DOI: <https://doi.org/10.1145/3035918.3064027>.
- [20] Wojciech Kazana. 2013. *Query evaluation with constant delay*. Theses. École normale supérieure de Cachan - ENS Cachan. <https://tel.archives-ouvertes.fr/tel-00919786>
- [21] François Le Gall. 2014. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation (ISSAC'14)*. ACM, New York., 296–303. DOI: <https://doi.org/10.1145/2608628.2608664>
- [22] Andrea Lincoln, Virginia Vassilevska Williams, and R. Ryan Williams. 2018. Tight hardness for shortest cycles and paths in sparse graphs. In *Proceedings of the 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2018)*. 1236–1252.
- [23] Bernard M. E. Moret and Henry D. Shapiro. 1991. *Algorithms from P to NP: Volume 1: Design & Efficiency*. Benjamin-Cummings.

- [24] Matthias Niewerth and Luc Segoufin. 2018. Enumeration of MSO queries on strings with constant delay and logarithmic updates. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (Houston, TX, , June 10-15, 2018)*, Jan Van den Bussche and Marcelo Arenas (Eds.). ACM, 179–191. DOI : <https://doi.org/10.1145/3196959.3196961>
- [25] Nicole Schweikardt, Luc Segoufin, and Alexandre Vigny. 2018. Enumeration for FO queries over nowhere dense graphs. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (Houston, TX, , June 10-15, 2018)*, Jan Van den Bussche and Marcelo Arenas (Eds.). ACM, 151–163. DOI : <https://doi.org/10.1145/3196959.3196971>
- [26] Luc Segoufin. 2015. Constant delay enumeration for conjunctive queries. *SIGMOD Rec.* 44, 1 (May 2015), 10–17. DOI : <https://doi.org/10.1145/2783888.2783894>
- [27] Luc Segoufin and Alexandre Vigny. 2017. Constant delay enumeration for FO queries over databases with local bounded expansion. In *ICDT 2017 (Leibniz International Proceedings in Informatics (LIPIcs))*, Michael Benedikt and Giorgio Orsi (Eds.), Vol. 68. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 20:1–20:16.
- [28] Yann Strobecki. 2010. *Enumeration complexity and matroid decomposition*. Ph.D. Dissertation. Université Paris Diderot - Paris 7.
- [29] Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *Proceedings of the 7th International Conference on Very Large Data Bases - Volume 7 (VLDB'81)*. VLDB Endowment, 82–94. <http://dl.acm.org/citation.cfm?id=1286831.1286840>.

Received March 2020; revised October 2020; accepted February 2021